

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Analiza imaginilor cu fund de ochi pentru ajutor în
diagnosticul retinopatiei diabetice**

Lucrare de dizertație

Prezentată ca cerință parțială pentru obținerea
titlului de *Master*
în domeniul *Calculatoare și Tehnologia Informației*
programul de studii *Ingineria Informației*

Conducător științific
Conf. dr. ing. Laura Maria FLOREA

Absolvent
Stancovici Marian

Anul 2026

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Analiza imaginilor cu fund de ochi pentru ajutor în diagnosticul retinopatiei diabetice*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Master* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria Informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Stancovici Marian

.....

Cuprins

Lista figurilor	iv
Lista tabelelor	v
Lista acronimelor	vii
1. Introducere	1
2. Fundamente teoretice	4
2.1. Diabetul zaharat și complicațiile sale oculare	4
2.2. Anatomia ochiului și retina	5
2.3. Retinopatia diabetică – definiție, stadii de evoluție și simptome	7
2.4. Imagistica de fund de ochi (”fundus photography”)	8
2.5. Diagnosticul clinic – procesul actual și limitările sale	10
2.6. Învățarea automată și învățarea profundă în analiza imaginilor medicale	11
2.7. Rețele neuronale convoluționale	13
2.8. Transfer learning și modele pre-antrenate	14
3. Stadiul actual al cercetării	16
3.1. Analiza seturilor de date disponibile	16
3.2. Metoda bazată pe arhitectura Inception-V3 [1]	18
3.3. “Transfer Learning” pentru diagnosticul retinopatiei diabetice [2]	19
3.4. “Multi-task Learning” pentru gradarea retinopatiei diabetice [3]	20
3.5. Analiza arhitecturii “Modified U-Net” pentru segmentarea semantică [4]	21
3.6. Tranziția către arhitecturi globale: RTNet (Relation Transformer Network) [5]	22
3.7. Arhitectura “Dual Branch” [6]	24
3.8. Abordarea “Explainable AI” [7]	24
3.9. Generalizarea în scenarii reale: “Multi-Model Domain Adaptation” [8]	25
3.10. Concluzii	26
4. Setul de date și preprocesarea imaginilor	27
4.1. Descrierea setului de date utilizat	27
4.2. Distribuția claselor	28
4.3. Probleme specifice imaginilor medicale	29
4.4. Etapele de preprocesare	30
4.5. Augmentarea datelor	31
4.6. Împărțirea datelor în seturi de antrenare, validare și testare	31
5. Metodologia propusă	33
5.1. Fluxul general al sistemului	33
5.2. Modelele analizate	33
5.3. Arhitectura ResNet50	33

5.4.	Arhitectura InceptionV3	34
5.5.	Arhitectura Inception-ResNet-V2	34
5.6.	Funcții de pierdere utilizate	34
5.7.	Optimizatorul și hiperparametrii de antrenare	35
5.8.	Strategia de evaluare	35
5.9.	Implementarea procesului de antrenare	35
6.	Rezultate experimentale	37
6.1.	Configurația experimentală	37
6.2.	Evoluția funcției de pierdere	38
6.3.	Evoluția acurateții pe setul de validare	38
6.4.	Analiza scorului AUC	38
6.5.	Analiza curbelor ROC multiclasă	39
6.6.	Analiza matricelor de confuzie	39
6.7.	Comparație între modelele testate	39
6.8.	Interpretarea erorilor de clasificare	39
7.	Aplicația de inferență	40
7.1.	Scopul aplicației dezvoltate	40
7.2.	Arhitectura aplicației	40
7.3.	Componenta backend	40
7.4.	Componenta frontend	40
7.5.	Fluxul de utilizare	41
7.6.	Integrarea modelului antrenat	41
7.7.	Limitări practice ale aplicației	41
8.	Concluzii și direcții viitoare	42
8.1.	Concluzii generale	42
8.2.	Rezultatele obținute față de obiectivele inițiale	42
8.3.	Limitările lucrării	42
8.4.	Direcții viitoare	43
	Bibliografie	45
	Anexa A. Cod	47

Lista figurilor

2.1.	Exemple de imagini de fund de ochi utilizate în evaluarea retinopatiei diabetice [7]	6
2.2.	Reprezentare schematică a progresiei claselor de severitate pentru retinopatia diabetică	8
2.3.	Flux conceptual pentru clasificarea imaginilor de fund de ochi cu o rețea neuronală convoluțională	12
3.1.	Exemple de imagini ale fundului de ochi în seturile de date DIARETDB0 și DIARETDB1 [7]	17
3.2.	Framework propus pentru detectarea și clasificarea retinopatiei diabetice [2] . . .	19
3.3.	Arhitectura U-net modificată [4]	22
3.4.	Curbele de Loss (stânga) și curbele Precision-Recall (dreapta) pentru studiile de ablație asupra celor patru leziuni DR. [5]	23
4.1.	Distribuția imaginilor pe clase și subseturi în <i>Diabetic_Balanced_Data</i>	28

Lista tabelelor

2.1. Interpretarea orientativă a claselor de severitate folosite în clasificarea retinopatiei diabetice	7
3.1. Tabel comparativ între seturile de date disponibile	18
4.1. Distribuția imaginilor în setul de date <i>Diabetic_Balanced_Data</i>	28
6.1. Experimente extinse adăugate pentru evaluare pe seturi de test separate	37

Lista acronimelor

SOTA = State of the Art

Deep DRiD = Deep Diabetic Retinopathy Image Dataset

UoA-DR = University of Auckland Diabetic Retinopathy

IDRiD = Indian Diabetic Retinopathy Image Dataset

DDR = Dataset of Diabetic Retinopathy

DRIVE = Digital Retinal Images for Vessel Extraction

Capitolul 1

Introducere

Contextul și motivația lucrării

Retinopatia diabetică reprezintă o provocare majoră de sănătate, fiind principala cauză a orbirii prevenibile în rândul populației active. Creșterea continuă a numărului de pacienți cu diabet generează presiune asupra sistemelor medicale, depășind adesea capacitatea de diagnosticare a specialiștilor. Din cauză că această afecțiune poate evolua asimptomatic în fazele inițiale, identificarea pacienților la risc este critică înainte ca leziunile retiniene să devină ireversibile, subliniind necesitatea unor soluții de monitorizare scalabile și eficiente.

Complexitatea diagnosticării este dată de necesitatea unei clasificări precise, conform standardelor internaționale care împart boala în cinci clase de severitate [3], [6]. Procesul impune medicului să distingă între stadiile incipiente, marcate doar de microanevrisme discrete [9], și formele avansate, caracterizate de hemoragii și neovascularizație. Această granularitate a diagnosticului este dificilă de menținut constant în programele de "screening" în masă, unde oboseala și subiectivitatea specialiștilor pot duce la erori de clasificare cu impact asupra deciziilor terapeutice.

Pentru a depăși aceste limitări, domeniul imagisticii medicale a adoptat soluții bazate pe inteligență artificială. Trecerea de la metodele clasice de procesare a imaginii [10] la algoritmi de învățare profundă ("Deep Learning") a marcat un punct de inflexiune, validat clinic prin studii extensive [1]. Rețelele neurale convoluționale au eliminat necesitatea definirii manuale a regulilor vizuale [11], având capacitatea de a învăța trăsături complexe direct din datele brute [7] și de a identifica corelații subtile între texturi și patologia ochiului.

Evoluția recentă a acestor tehnologii a permis tranziția de la modele experimentale simple la arhitecturi robuste [5], capabile să opereze în condiții clinice reale. Între timp cercetarea actuală s-a distanțat de clasificarea binară elementară, concentrându-se pe sisteme care pot gestiona diversitatea imaginilor provenite din multiple surse prin tehnici de adaptare la domeniu [8]. Algoritmi moderni demonstrează o flexibilitate crescută la artefacte și zgomot vizual, oferind o consistență a diagnosticului care reușesc să atingă nivelul expertizei umane [7], [2].

Obiectivele lucrării

Scopul lucrării de față este de a folosi algoritmi de vedere computerizată bazați pe rețele convoluționale adânci pentru a crea o metodă de detectare și clasificare a retinopatiei diabetice. Metoda se va testa pe seturi de date publice. Implementarea se va face în Python folosind biblioteci specifice.

Aceasta urmărește atingerea următoarelor obiective:

- Analizarea metodelor deja propuse în literatura de specialitate pentru detecția și clasificarea automată a retinopatiei diabetice folosind imagini cu fund de ochi.

- Căutarea seturilor de date publice ce conțin imagini cu fund de ochi și adnotări pentru clasificarea retinopatiei diabetice.
- Analizarea seturilor de date găsite.
- Propunerea unei metode bazate pe învățare profundă pentru detecția și clasificarea retinopatiei diabetice.
- Analizarea performanței pentru metoda propusă în diverse cazuri de antrenare/testare

Structura lucrării

Lucrarea este organizată în opt capitole principale, urmărind trecerea de la contextul medical și stadiul actual al cercetării la metodologia propusă, rezultatele experimentale și aplicația de inferență dezvoltată.

- **Capitolul 1 – Introducere:** Se prezintă contextul medical al retinopatiei diabetice, motivația temei, obiectivele lucrării și modul de organizare a documentului.
- **Capitolul 2 – Fundamente teoretice:** Se introduc noțiunile medicale și tehnice necesare: diabetul zaharat, retina, stadiile retinopatiei diabetice, imaginile de fund de ochi, rețelele neuronale convoluționale și principiile învățării profunde.
- **Capitolul 3 – Stadiul actual al cercetării:** Se analizează metodele existente pentru detecția și clasificarea automată a retinopatiei diabetice, cu accent pe arhitecturi CNN, "transfer learning" și tehnici de generalizare între seturi de date.
- **Capitolul 4 – Setul de date și preprocesarea imaginilor:** Se descriu datele utilizate, distribuția claselor, problemele de dezechilibru, etapele de preprocesare, augmentarea imaginilor și împărțirea în seturi de antrenare, validare și testare.
- **Capitolul 5 – Metodologia propusă:** Se prezintă fluxul general al sistemului, modelele analizate, arhitecturile ResNet50, InceptionV3, (poate Efficientnet_b3) și Inception-ResNet-V2, funcțiile de pierdere utilizate, optimizatorul, hiperparametrii și strategia de evaluare.
- **Capitolul 6 – Rezultate experimentale:** Include analiza curbelor de antrenare, a acurateții, a scorului AUC, a curbelor ROC multiclasă și a matricelor de confuzie, precum și comparația între configurațiile testate.
- **Capitolul 7 – Aplicația de inferență:** Descrie aplicația web dezvoltată pentru testarea modelului, arhitectura frontend-backend, fluxul de utilizare și integrarea modelului antrenat.
- **Capitolul 8 – Concluzii și direcții viitoare:** Se sintetizează rezultatele obținute, contribuțiile lucrării, limitările identificate și posibilele direcții de dezvoltare ulterioară.

Capitolul 2

Fundamente teoretice

2.1 Diabetul zaharat și complicațiile sale oculare

Diabetul zaharat este o afecțiune metabolică cronică definită prin menținerea unor valori crescute ale glicemiei pe perioade lungi. În mod fiziologic, nivelul glucozei din sânge este reglat în principal prin acțiunea insulinei, hormon produs de pancreas, care facilitează absorbția glucozei de către celule și utilizarea acesteia ca sursă de energie. În cazul diabetului, acest mecanism este perturbat fie prin absența sau producerea insuficientă de insulină, fie prin rezistența țesuturilor la acțiunea acesteia. Din punct de vedere clinic, cele mai cunoscute forme sunt diabetul zaharat de tip 1, asociat cu distrugerea celulelor beta pancreatice și deficit insulenic sever, și diabetul zaharat de tip 2, asociat mai ales cu rezistența la insulină și cu factori metabolici precum obezitatea, sedentarismul și predispoziția genetică [12].

Importanța diabetului pentru această lucrare nu se limitează la caracterul său metabolic, ci derivă în special din complicațiile pe care le produce. Hiperglicemia cronică afectează vasele de sânge de calibru mic și mare, nervii periferici, rinichii, cordul și structurile oculare. Complicațiile microvasculare sunt de interes major în oftalmologie, deoarece retina este un țesut puternic vascularizat și sensibil la variațiile aportului de oxigen și nutrienți. Vasele retiniene au un diametru redus, iar peretele vascular poate fi deteriorat gradual de modificările biochimice asociate diabetului. În timp, apar alterări ale permeabilității vasculare, microanevrisme, hemoragii, exudate, zone ischemice și, în stadiile avansate, vase de neoformație.

Retina este una dintre puținele regiuni ale organismului în care microcirculația poate fi observată direct prin imagistică neinvazivă. Această particularitate transformă examinarea fundului de ochi într-un instrument esențial pentru monitorizarea efectelor diabetului asupra sistemului vascular. Din perspectivă medicală, modificările retiniene reflectă atât afectarea oculară locală, cât și severitatea afectării microvasculare. Din perspectivă computațională, imaginile retiniene oferă un suport vizual bogat, în care semnele patologice pot fi analizate prin metode automate de procesare a imaginilor și prin modele de învățare profundă.

Complicațiile oculare ale diabetului includ retinopatia diabetică, edemul macular diabetic, cataracta și glaucomul. Dintre acestea, retinopatia diabetică este cea mai importantă în contextul "screening-ului" automat, deoarece evoluează de obicei lent, poate fi asimptomatică în fazele inițiale și poate produce pierdere ireversibilă de vedere dacă nu este identificată și tratată la timp. Edemul macular diabetic este o complicație strâns legată de retinopatie și apare atunci când permeabilitatea vasculară crescută determină acumularea de lichid în zona maculei, regiunea responsabilă pentru vederea centrală fină [13]. Chiar și atunci când retinopatia nu este în stadiu proliferativ, afectarea maculară poate reduce semnificativ acuitatea vizuală.

În contextul acestei lucrări, cele mai relevante complicații oculare pot fi sintetizate astfel:

- **Retinopatia diabetică** – afectează vasele retiniene și reprezintă principala țintă a sistemului automat de clasificare.

- **Edemul macular diabetic** – implică acumularea de lichid în zona maculei și poate reduce vederea centrală.
- **Cataracta și glaucomul** – nu sunt obiectivul direct al clasificării, dar pot influența calitatea imaginii și interpretarea clinică.

Din punctul de vedere al sistemelor automate, retinopatia diabetică este o problemă dificilă deoarece semnele bolii au dimensiuni, forme și contraste diferite. Unele leziuni, cum sunt microanevrismele, pot fi foarte mici și greu de separat de zgomotul imaginii sau de artefactele de achiziție. Alte leziuni, precum hemoragiile sau exudatele, sunt mai vizibile, dar pot varia mult în funcție de pacient, de calitatea fotografiei și de camera folosită. În plus, clasificarea clinică nu presupune doar detectarea existenței unor leziuni, ci estimarea gradului de severitate al bolii, ceea ce introduce o componentă ordinală: clasele nu sunt complet independente, ci descriu o progresie patologică.

Această progresie graduală explică de ce retinopatia diabetică este potrivită pentru o abordare de clasificare multiclasă. În multe seturi de date utilizate în cercetare, imaginile sunt împărțite în cinci clase: fără retinopatie diabetică, retinopatie ușoară, moderată, severă și proliferativă. O astfel de etichetare permite antrenarea unor algoritmi care nu se limitează la decizia binară sănătos/bolnav, ci estimează nivelul de risc și pot susține o prioritizare mai bună a pacienților. În practică, o eroare între două clase adiacente poate avea o semnificație diferită față de o eroare între clase îndepărtate. De exemplu, confundarea clasei 1 cu clasa 2 este mai puțin gravă decât confundarea unei forme proliferative cu o imagine fără semne patologice sau de severitate ușoară.

2.2 Anatomia ochiului și retina

Ochiul uman este un sistem optic complex, specializat în captarea luminii și transformarea acesteia în semnale nervoase interpretate de creier. Din punct de vedere funcțional, componentele principale implicate în formarea imaginii sunt corneea, cristalinul, umoarea apoasă, corpul vitros și retina. Corneea și cristalinul focalizează lumina, iar retina funcționează ca un strat fotosensibil situat în partea posterioară a globului ocular. În contextul analizei automate a imaginilor, retina este structura de interes, deoarece aici pot fi observate vasele sanguine, discul optic, macula și leziunile asociate retinopatiei diabetice.

Retina este formată din mai multe straturi celulare, incluzând fotoreceptori, celule bipolare, celule ganglionare și celule de suport. Fotoreceptorii transformă stimulii luminoși în semnale electrice, iar informația este transmisă prin nervul optic către cortexul vizual. Regiunea centrală a retinei, numită maculă, este responsabilă pentru vederea detaliată, necesară cititului, recunoașterii fețelor și percepției fine a culorilor. În centrul maculei se află foveea, zona cu cea mai mare densitate de conuri și cu acuitatea vizuală maximă. Din acest motiv, afectarea maculei prin edem sau exudate poate avea consecințe majore asupra calității vieții pacientului.

În imaginile de fund de ochi, câteva repere anatomice sunt esențiale pentru interpretare. Discul optic apare de obicei ca o regiune circulară sau ovalară, mai luminoasă, din care pornesc vasele retiniene principale. Vasele se ramifică progresiv și formează o rețea vasculară care acoperă suprafața retiniană vizibilă. Macula se observă ca o zonă mai întunecată, situată lateral față de discul optic, iar foveea reprezintă centrul acesteia. Pentru algoritmi de analiză, aceste structuri au rol dublu: pe de o parte oferă informație anatomică utilă, iar pe de altă

parte pot produce confuzii. De exemplu, discul optic este o regiune luminoasă care poate fi confundată cu exudatele dacă modelul nu învață contextul anatomic corect.

Principalele repere anatomice urmărite într-o imagine de fund de ochi sunt:

- **Discul optic** – regiune luminoasă din care pornesc vasele retiniene principale.
- **Macula** – zonă responsabilă pentru vederea centrală, importantă în evaluarea edemului macular.
- **Foveea** – centrul maculei, asociat cu acuitatea vizuală maximă.
- **Rețeaua vasculară** – structură relevantă pentru identificarea hemoragiilor, microanevrismelor și modificărilor ischemice.

Vasele retiniene au o importanță deosebită în detectarea modificărilor patologice. În diabet, afectarea microvasculară determină fragilizarea pereților capilari și modificări ale fluxului sanguin. Microanevrismele apar ca mici dilatații ale capilarelor, fiind printre cele mai timpurii semne vizibile ale retinopatiei diabetice. Hemoragiile sunt rezultatul ruperii sau scurgerii vaselor deteriorate, iar exudatele apar prin depunerea de lipide și proteine în urma creșterii permeabilității vasculare [14]. În stadiile avansate, ischemia retiniană poate stimula formarea de vase noi, fragile și anormale, fenomen numit neovascularizație.

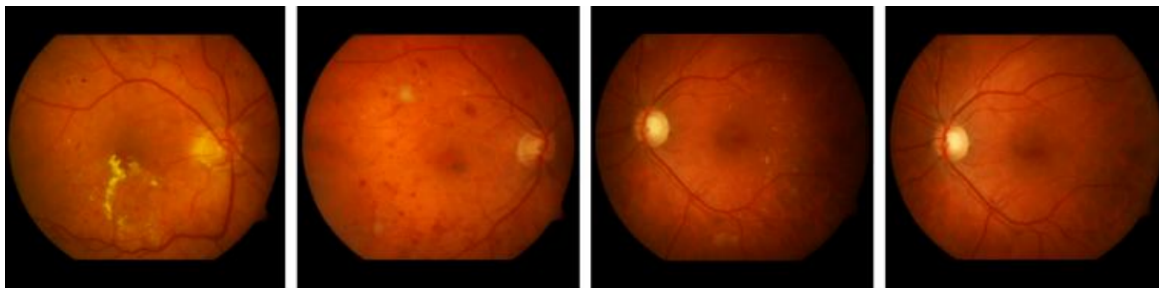


Figura 2.1: Exemple de imagini de fund de ochi utilizate în evaluarea retinopatiei diabetice [7]

Din perspectiva prelucrării imaginilor, retina prezintă o combinație de structuri globale și detalii locale fine. Structurile globale includ forma câmpului vizual, poziția discului optic, distribuția vaselor principale și variațiile generale de iluminare. Detaliile locale includ microanevrismele, punctele hemoragice, exudatele mici sau modificările subtile de textură. Această combinație justifică utilizarea rețelelor neuronale convoluționale, deoarece acestea pot extrage treptat trăsături de la nivel local la nivel global. Straturile inițiale pot învăța muchii, contraste și texturi, iar straturile profunde pot combina aceste informații pentru a recunoaște tipare patologice mai complexe.

Un alt aspect relevant este variabilitatea normală a aspectului retinei. Pigmentarea fundului de ochi diferă între pacienți, diametrul vaselor poate varia, iar poziția discului optic în imagine depinde de modul de achiziție. De asemenea, fotografia fundus poate include zone supraexpuse, reflexii, blur, margini întunecate sau câmp vizual incomplet. Această variabilitate face ca analiza automată să fie mai dificilă decât clasificarea unor obiecte fotografiate în condiții controlate. Modelele trebuie să distingă între variații anatomice normale, artefacte de imagine și semne patologice reale.

2.3 Retinopatia diabetică – definiție, stadii de evoluție și simptome

Retinopatia diabetică este o complicație microvasculară a diabetului zaharat, caracterizată prin afectarea progresivă a vaselor de sânge retiniene. Mecanismele patologice includ lezarea endoteliului vascular, pierderea pericitelor, creșterea permeabilității capilare, ocluzii vasculare și ischemie retiniană. În timp, aceste procese duc la apariția unor leziuni vizibile în imaginile fundului de ochi. În fazele incipiente, pacientul poate să nu observe nicio modificare a vederii, ceea ce face ca screening-ul periodic să fie esențial. În fazele avansate, boala poate produce vedere încețoșată, pete în câmpul vizual, scăderea acuității vizuale și, în cazuri severe, pierderea vederii.

Clasificarea clinică a retinopatiei diabetice este de obicei organizată pe niveluri de severitate. În lucrările de învățare automată, inclusiv în seturile de date publice folosite frecvent, se utilizează adesea o scară cu cinci clase: clasa 0 – fără retinopatie diabetică, clasa 1 – retinopatie diabetică ușoară, clasa 2 – retinopatie diabetică moderată, clasa 3 – retinopatie diabetică severă și clasa 4 – retinopatie diabetică proliferativă [1], [3], [15]. Această formulare este utilă pentru antrenarea modelelor de clasificare, deoarece transformă evaluarea clinică într-o problemă de predicție multclasă.

Tabela 2.1: Interpretarea orientativă a claselor de severitate folosite în clasificarea retinopatiei diabetice

Clasa	Grad de severitate	Semne vizuale urmărite în imagine
0	Fără retinopatie diabetică	Nu sunt observate leziuni specifice; imaginea poate fi considerată normală din punct de vedere al retinopatiei diabetice.
1	Retinopatie ușoară	Microanevrisme izolate, de obicei greu vizibile și ușor de confundat cu zgomot sau artefacte de achiziție.
2	Retinopatie moderată	Număr mai mare de leziuni, cu posibile hemoragii, exudate dure și modificări vasculare vizibile.
3	Retinopatie severă	Afectare vasculară extinsă, numeroase hemoragii și risc crescut de progresie către forma proliferativă.
4	Retinopatie proliferativă	Neovascularizație și risc major de complicații severe, precum hemoragie vitroasă sau dezlipire de retină.

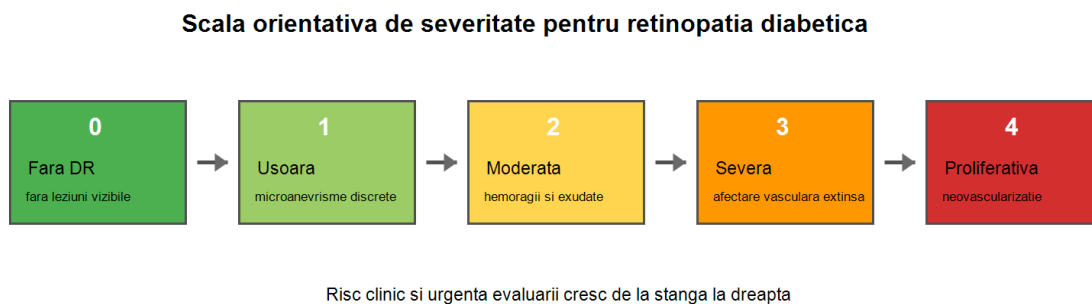


Figura 2.2: Reprezentare schematică a progresiei claselor de severitate pentru retinopatia diabetică

Clasele 0 și 1 sunt de obicei cele mai subtile din punct de vedere vizual. Absența semnelor evidente într-o fotografie nu garantează absența completă a riscului, deoarece detectabilitatea leziunilor depinde de calitatea imaginii și de câmpul vizual acoperit. În același timp, retinopatia ușoară poate fi reprezentată doar de câteva microanevrisme, motiv pentru care confuziile între clasele 0, 1 și 2 sunt frecvente în modelele automate.

Clasele 3 și 4 sunt asociate cu afectare vasculară mai evidentă și cu risc clinic mai ridicat. Imaginea devine, în general, mai evident patologică, iar modelele automate tind să obțină performanțe mai bune pentru aceste stadii decât pentru cele incipiente. Totuși, identificarea corectă a claselor avansate este critică, deoarece subestimarea severității poate întârzia intervenția medicală. Într-un sistem de screening, o eroare de tip fals negativ pentru o formă proliferativă este mult mai gravă decât o confuzie între două clase adiacente.

În afara clasificării pe stadii, este importantă și natura leziunilor observabile. Cele mai importante semne patologice sunt:

- **Microanevrismele** – printre primele semne ale bolii, vizibile ca puncte roșii foarte mici.
- **Hemoragiile** – pot avea forme variate, de la punctiforme la pete mai extinse.
- **Exudatele dure** – regiuni galben-albicioase, bine delimitate, asociate cu scurgeri lipidice.
- **Exudatele moi** – pete vătămate care indică zone de ischemie în stratul fibrelor nervoase.
- **Neovascularizația** – semn al trecerii către forma proliferativă.

Metodele automate pot aborda aceste semne fie prin clasificare globală a imaginii, fie prin segmentarea leziunilor, fie prin combinații multi-task, în care modelul învață simultan mai multe sarcini legate de severitate și localizare [3], [5].

2.4 Imagistica de fund de ochi ("fundus photography")

Fotografia de fund de ochi reprezintă una dintre cele mai utilizate metode de examinare imagistică a retinei. Aceasta permite captarea unei imagini color a suprafeței retiniene, incluzând discul optic, vasele sanguine, macula și eventualele leziuni patologice. Spre deosebire de metode mai complexe, precum angiografia cu fluoresceină sau tomografia în coerență optică, fotografia fundus este relativ rapidă, neinvazivă și potrivită pentru programe de screening la

scară largă [1]. Din acest motiv, majoritatea seturilor de date utilizate în cercetarea privind detecția automată a retinopatiei diabetice se bazează pe astfel de imagini.

În mod obișnuit, o imagine fundus este obținută cu o cameră specializată care iluminează retina și captează lumina reflectată prin pupilă. Achiziția poate fi realizată cu sau fără dilatarea pupilei, iar câmpul vizual poate varia în funcție de echipament. Imaginile pot avea rezoluții diferite, niveluri diferite de contrast și grade variabile de claritate. În plus, pot apărea artefacte precum reflexii luminoase, umbre, zone întunecate, margini circulare, praf pe lentilă sau defocalizare. Toate aceste elemente influențează atât evaluarea umană, cât și performanța algoritmilor de clasificare.

Pentru analiza automată, imaginea fundus prezintă mai multe provocări. Prima este variația de iluminare. Unele imagini sunt supraexpuse, iar leziunile luminoase pot fi greu de distins de fundal. Altele sunt subexpuse, iar leziunile roșii sau vasele fine pot deveni slab vizibile. A doua provocare este variația cromatică: tonul general al retinei diferă între pacienți și între camere. A treia provocare este rezoluția foarte mare a imaginilor originale. Leziunile mici pot ocupa doar câțiva pixeli, iar redimensionarea agresivă poate elimina informație relevantă. În același timp, procesarea directă a imaginilor la rezoluție completă poate fi costisitoare computațional.

Cele mai frecvente dificultăți întâlnite în imaginile de fund de ochi sunt:

- variații de iluminare și contrast între imagini;
- diferențe cromatice între pacienți și între camere de achiziție;
- rezoluții diferite și dimensiuni mari ale imaginilor originale;
- artefacte precum reflexii, blur, margini întunecate sau câmp vizual incomplet;
- dimensiunea foarte mică a unor leziuni, în special a microanevrismelor.

Preprocesarea imaginilor are rolul de a reduce aceste variații și de a oferi modelului date mai consistente. Etapele frecvente includ decuparea marginilor negre, redimensionarea la o dimensiune standard, normalizarea valorilor pixelilor, corectarea iluminării și aplicarea unor transformări de augmentare. În unele abordări, se utilizează metode pentru evidențierea vaselor sau pentru îmbunătățirea contrastului local. În altele, modelul primește imaginea aproape brută, iar rețeaua neuronală învață singură trăsăturile relevante. Alegerea depinde de dimensiunea setului de date, de arhitectura folosită și de obiectivul experimentului.

Într-un flux tipic de lucru, preprocesarea poate include:

- decuparea zonelor negre din jurul câmpului circular al imaginii;
- redimensionarea imaginilor la dimensiunea cerută de model;
- normalizarea intensităților pixelilor;
- corectarea parțială a iluminării neuniforme;
- augmentări precum rotații, flip-uri, modificări de luminozitate sau contrast.

Din punct de vedere practic, preprocesarea trebuie să păstreze leziunile fine, nu doar să uniformizeze aspectul general al imaginii. O redimensionare prea agresivă poate elimina microanevrismele, iar o corecție de contrast nepotrivită poate accentua artefactele. De aceea, etapele de preprocesare sunt alese în funcție de modelul folosit și de rezoluția minimă la care semnele patologice rămân vizibile.

Înainte de apariția metodelor moderne de învățare profundă, multe sisteme de detecție automată se bazau pe extragerea manuală a trăsăturilor. De exemplu, se încerca identificarea leziunilor roșii, a exudatelor sau a vaselor prin praguri de culoare, filtre, segmentare și clasificatori clasici. Astfel de metode au contribuit la dezvoltarea domeniului și au demonstrat că semnele retinopatiei pot fi detectate automat [16], [10], [9]. Totuși, ele erau sensibile la variațiile de achiziție și necesitau proiectarea manuală a regulilor vizuale. Rețelele neuronale convoluționale au redus această dependență, învățând reprezentări direct din date [11].

Un element important în folosirea imaginilor fundus pentru clasificare este raportul dintre informația globală și cea locală. O imagine poate conține semne patologice mici, dar decizia finală depinde și de distribuția acestora în câmpul retinian. Un model care analizează doar fragmente locale poate detecta leziuni, dar poate pierde contextul global. Un model care analizează imaginea întreagă poate învăța distribuția generală, dar poate rata detalii fine după redimensionare. Din acest motiv, literatura recentă explorează arhitecturi care combină informații la mai multe scări, ramuri multiple sau mecanisme de atenție [6], [5].

2.5 Diagnosticul clinic – procesul actual și limitările sale

Diagnosticul clinic al retinopatiei diabetice se bazează pe evaluarea imaginilor retiniene de către medici oftalmologi sau specialiști instruiți în screening. În cadrul unui flux standard, pacientului i se realizează una sau mai multe fotografii ale fundului de ochi, iar imaginile sunt analizate pentru identificarea leziunilor specifice. În funcție de severitate, pacientul poate fi încadrat într-o categorie de risc și poate primi recomandări pentru monitorizare, investigații suplimentare sau tratament. În programele de screening, scopul principal este identificarea pacienților care necesită evaluare medicală mai rapidă, înainte de apariția pierderii ireversibile de vedere.

Procesul clinic are însă mai multe limitări:

- **Disponibilitatea specialiștilor.** Pe măsură ce numărul pacienților cu diabet crește, volumul imaginilor care trebuie evaluate devine tot mai mare.
- **Variabilitatea inter-observator.** Doi specialiști pot evalua diferit aceeași imagine, mai ales în cazurile de graniță dintre clase.
- **Variabilitatea intra-observator.** Același specialist poate avea variații în decizie în funcție de oboseală, experiență sau calitatea imaginii.
- **Calitatea imaginilor.** Imaginile slab focalizate, întunecate sau parțial acoperite de artefacte sunt greu de interpretat.

În practică, problema calității imaginii rămâne deosebit de importantă. În unele cazuri, imaginea trebuie repetată, ceea ce crește timpul și costul examinării. În alte cazuri, imaginea poate fi evaluată cu incertitudine, ceea ce afectează consistența diagnosticului. Din acest motiv,

unele sisteme moderne includ nu doar clasificarea retinopatiei, ci și estimarea calității imaginii, astfel încât imaginile neinterpretabile să fie semnalate separat [15].

Sistemele automate bazate pe inteligență artificială nu înlocuiesc medicul, dar pot funcționa ca instrumente de suport. Într-un scenariu de screening, un algoritm poate analiza rapid un număr mare de imagini și poate prioritiza cazurile suspecte. De asemenea, poate oferi o a doua opinie sau poate reduce timpul necesar pentru trierea imaginilor fără semne evidente. Studiile moderne au demonstrat că modelele de învățare profundă pot atinge performanțe ridicate în detectarea retinopatiei diabetice pe imagini fundus [1], [17]. Totuși, integrarea clinică necesită validare riguroasă, interpretabilitate, robustețe la imagini din surse diferite și control al erorilor critice.

O dificultate specifică diagnosticului automat este faptul că nu toate erorile au aceeași importanță. În clasificarea multiclasă, o predicție greșită cu o clasă adiacentă poate fi tolerabilă în anumite contexte, dar o predicție care subestimează severitatea bolii poate întârzia tratamentul. De exemplu, clasificarea unei imagini severe ca fiind fără retinopatie este o eroare mult mai periculoasă decât clasificarea unei imagini moderate ca ușoare. Din acest motiv, evaluarea unui model nu trebuie să se bazeze exclusiv pe acuratețe globală, ci pe un set mai larg de metrici:

- **sensibilitate** – capacitatea de a identifica imaginile patologice;
- **specificitate** – capacitatea de a evita alarmele false;
- **scor AUC** – calitatea separării între clase pentru diferite praguri;
- **matrice de confuzie** – distribuția erorilor între clase;
- **performanță pe fiecare clasă** – utilă mai ales în seturi de date dezechilibrate.

2.6 Învățarea automată și învățarea profundă în analiza imaginilor medicale

Învățarea automată este un domeniu al inteligenței artificiale care urmărește dezvoltarea unor algoritmi capabili să învețe modele din date. În loc ca regulile să fie definite explicit de programator, sistemul ajustează parametri interni pe baza unor exemple. În cazul clasificării imaginilor medicale, exemplele sunt imagini asociate cu etichete clinice, iar obiectivul modelului este să învețe o funcție care primește o imagine și produce o predicție. Pentru retinopatia diabetică, această predicție poate fi binară, de tipul retinopatie prezentă/absentă, sau multiclasă, corespunzătoare severității bolii [1], [17].

Metodele clasice de învățare automată necesitau de obicei o etapă separată de extragere a trăsăturilor. În imaginile retiniene, aceste trăsături puteau include culoarea, textura, forma leziunilor, grosimea vaselor sau distribuția spațială a exudatelor. După extragerea trăsăturilor, un clasificator precum SVM, k-NN sau Random Forest putea fi antrenat pentru a lua decizia finală. Avantajul acestor metode este interpretabilitatea relativ mai mare a unor trăsături proiectate manual. Dezavantajul este dependența puternică de calitatea acestor trăsături și dificultatea de a acoperi toată variabilitatea imaginilor reale.

Într-o abordare clasică, pipeline-ul era de obicei împărțit în etape distincte:

- preprocesarea imaginii și corectarea iluminării;

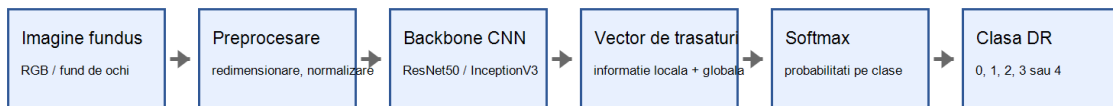
- extragerea manuală a trăsăturilor relevante;
- selectarea sau reducerea dimensionalității trăsăturilor;
- antrenarea unui clasificator clasic;
- validarea rezultatului pe imagini nefolosite la antrenare.

Învățarea profundă schimbă această paradigmă prin folosirea unor rețele neuronale cu multe straturi, capabile să învețe reprezentări ierarhice [18]. Într-o rețea convoluțională, primele straturi învață filtre simple, precum muchii, contraste locale sau texturi. Straturile intermediare combină aceste filtre în forme mai complexe, iar straturile profunde pot reprezenta structuri relevante pentru sarcina finală. Această ierarhie este potrivită pentru imagini medicale, deoarece semnele patologice apar la scări diferite și în contexte anatomice variate.

În procesul de antrenare supervizată, modelul primește perechi imagine-etichetă și produce o predicție. Diferența dintre predicție și eticheta reală este măsurată printr-o funcție de pierdere, iar parametrii rețelei sunt actualizați prin algoritmi de optimizare, de obicei pe baza propagării inverse a gradientului. Pentru clasificarea multiclasă, una dintre cele mai folosite funcții de pierdere este Cross-Entropy. În situații cu dezechilibru între clase, se pot folosi variante ponderate, în care erorile pe clasele rare au o importanță mai mare. O altă opțiune este Focal Loss, care pune accent pe exemplele dificile și reduce contribuția exemplorlor clasificate ușor [19].

Dezechilibrul claselor este o problemă frecventă în retinopatia diabetică. În multe seturi de date, clasa fără retinopatie este mult mai numeroasă decât clasele severe. Dacă modelul este antrenat fără corecții, acesta poate obține acuratețe bună prin favorizarea clasei dominante, dar poate avea sensibilitate slabă pentru clasele rare. Din acest motiv, evaluarea trebuie să urmărească performanța pe fiecare clasă, nu doar media globală. Tehnicile de reechilibrare includ ponderarea funcției de pierdere, eșantionarea echilibrată, augmentarea claselor minoritare sau folosirea unor metrice robuste la dezechilibru.

Flux general pentru clasificarea imaginilor fundus cu CNN



Transfer learning: modelul porneste de la greutatea pre-antrenate si este ajustat pe imaginile retiniene.

Figura 2.3: Flux conceptual pentru clasificarea imaginilor de fund de ochi cu o rețea neuronală convoluțională

În analiza imaginilor medicale, o altă provocare este dimensiunea limitată a datelor adnotate. Obținerea etichetelor clinice necesită timp și expertiză, iar datele pot fi afectate de restricții etice și de confidențialitate. Transfer learning-ul este o soluție practică pentru această

problemă. În loc ca modelul să fie antrenat de la zero, se pornește de la o arhitectură pre-antrenată pe un set mare de imagini generale, iar apoi modelul este adaptat la sarcina medicală. Această abordare poate îmbunătăți convergența și performanța, mai ales atunci când setul de date medical nu este foarte mare [2].

2.7 Rețele neuronale convoluționale

Rețelele neuronale convoluționale, cunoscute sub abrevierea CNN, reprezintă una dintre cele mai importante arhitecturi pentru analiza imaginilor. Ele sunt concepute pentru a exploata structura spațială a datelor vizuale. Spre deosebire de o rețea complet conectată, care ar trata fiecare pixel independent de poziția sa, o rețea convoluțională aplică filtre locale pe imagine și păstrează relațiile spațiale dintre pixeli. Această proprietate este esențială pentru imagini retiniene, deoarece semnele patologice sunt definite de configurații locale de culoare, formă și textură [11].

Operația de convoluție constă în deplasarea unui filtru peste imagine și calcularea unui răspuns local. Fiecare filtru învață să detecteze un anumit tipar vizual. În primele straturi, filtrele pot detecta margini, pete sau contraste. În straturile mai profunde, combinațiile de filtre pot detecta structuri mai complexe, precum vase, exudate sau zone hemoragice. Parametrii filtrelor sunt învățați automat în timpul antrenării, ceea ce elimină necesitatea proiectării manuale a descriptorilor vizuali.

Pe lângă straturile convoluționale, CNN-urile includ mai multe componente recurente:

- **funcții de activare**, care introduc neliniaritate și permit modelului să aproximeze relații complexe;
- **operații de pooling**, care reduc dimensiunea spațială a hărților de trăsături;
- **straturi de normalizare**, care pot stabiliza antrenarea și accelera convergența;
- **straturi complet conectate sau globale**, care transformă trăsăturile extrase într-o predicție finală;
- **strat softmax**, folosit frecvent pentru a obține probabilități pe clase în clasificarea mult-clasă.

Arhitecturile CNN moderne diferă prin adâncime, tipul blocurilor folosite și modul în care gestionează propagarea informației. ResNet introduce conexiuni reziduale, care permit antrenarea unor rețele mai adânci prin transmiterea directă a informației între blocuri [20]. Această idee reduce problema degradării performanței în rețele foarte adânci și facilitează învățarea unor reprezentări complexe. Inception folosește blocuri cu filtre de dimensiuni diferite în paralel, pentru a captura informație la mai multe scări [21]. Acest lucru este relevant în imaginile retiniene, unde leziunile pot varia de la puncte foarte mici la regiuni extinse.

În retinopatia diabetică, CNN-urile pot fi folosite în mai multe moduri:

- **clasificare globală**, în care modelul primește fotografia fundus și produce gradul de severitate;
- **detectie de leziuni**, în care modelul indică prezența unor semne patologice;

- **segmentare semantică**, în care modelul identifică regiunile afectate la nivel de pixel;
- **modele hibride sau multi-task**, care învață simultan clasificarea, localizarea și interpretarea leziunilor.

Segmentarea poate oferi informație mai detaliată și poate susține interpretabilitatea, dar necesită etichete pixel-level, mai greu de obținut. Metodele hibride combină ramuri globale și locale pentru a integra contextul anatomic cu detaliile patologice [3], [6].

Un aspect important al CNN-urilor este interpretabilitatea. Deși performanța lor poate fi ridicată, modelele profunde sunt adesea percepute ca sisteme de tip “cutie neagră”. În medicină, această problemă este critică, deoarece deciziile trebuie să fie verificabile și explicabile. Tehnici precum hărțile de activare a claselor pot evidenția regiunile care au influențat predicția, oferind medicului o indicație vizuală asupra motivului pentru care modelul a ales o anumită clasă. Astfel de metode nu garantează corectitudinea deciziei, dar pot crește transparența și pot ajuta la detectarea cazurilor în care modelul se bazează pe artefacte sau regiuni irelevante [7].

2.8 Transfer learning și modele pre-antrenate

Transfer learning-ul este o tehnică prin care cunoștințele învățate într-o sarcină sunt re-utilizate pentru o altă sarcină. În contextul imaginilor medicale, această tehnică este foarte importantă deoarece seturile de date adnotate sunt mai greu de obținut decât în domeniile generale ale vederii computerizate. Un model pre-antrenat pe milioane de imagini poate învăța filtre generale pentru muchii, texturi, forme și compoziții vizuale. Chiar dacă imaginile naturale diferă de imaginile retiniene, primele straturi ale rețelei pot rămâne utile, iar straturile finale pot fi adaptate la clasificarea medicală.

Există mai multe moduri de utilizare a transfer learning-ului:

- **Extractor de trăsături fix.** Straturile convoluționale sunt păstrate fixe, iar doar clasificatorul final este antrenat pe datele medicale.
- **Fine-tuning parțial.** Se ajustează doar ultimele blocuri ale rețelei, pentru a adapta reprezentările la imaginile retiniene.
- **Fine-tuning complet.** Toți parametrii modelului sunt actualizați, variantă utilă când există suficiente date și regularizare adecvată.

Fine-tuning-ul poate oferi performanțe mai bune, dar necesită atenție la rata de învățare, dimensiunea setului de date, regularizare și optimizatorul folosit, Adam fiind una dintre alegerile frecvente în antrenarea rețelelor profunde [22].

Modelele precum ResNet50 și InceptionV3 sunt alegeri frecvente pentru clasificarea imaginilor medicale deoarece oferă un compromis bun între capacitatea de reprezentare și costul computațional. ResNet50 este suficient de adânc pentru a extrage trăsături complexe, iar conexiunile reziduale susțin antrenarea stabilă. InceptionV3 folosește procesare multi-scală, utilă pentru leziuni de dimensiuni diferite. În aplicații de retinopatie diabetică, aceste arhitecturi pot fi adaptate prin înlocuirea ultimului strat cu un clasificator pentru cele cinci clase de severitate și prin antrenarea pe imaginile fundus disponibile.

Transfer learning-ul nu elimină însă toate problemele. Dacă datele de antrenare sunt dezechilibrate sau conțin artefacte sistematice, modelul poate învăța corelații nedorite. De exemplu,

dacă imaginile severe provin predominant de la o anumită cameră sau au o anumită rezoluție, modelul poate asocia caracteristicile tehnice cu boala. Această problemă este cunoscută ca bias de domeniu și afectează capacitatea de generalizare. Din acest motiv, este importantă evaluarea pe seturi de validare și testare reprezentative, precum și analiza erorilor prin matrice de confuzie și curbe ROC.

În această lucrare, fundamentele prezentate în capitolul curent susțin partea experimentală ulterioară. Retinopatia diabetică este o problemă medicală cu impact ridicat, imaginile de fund de ochi oferă suportul vizual pentru screening automat, iar rețelele convoluționale permit învățarea directă a trăsăturilor relevante. Transfer learning-ul facilitează adaptarea arhitecturilor moderne la date medicale, iar metricile de evaluare permit compararea configurațiilor de antrenare. Împreună, aceste elemente definesc cadrul teoretic necesar pentru metodologia propusă și pentru interpretarea rezultatelor experimentale.

Capitolul 3

Stadiul actual al cercetării

3.1 Analiza seturilor de date disponibile

Performanța și capacitatea de generalizare a modelelor de învățare profundă sunt intrinsec dependente de calitatea, volumul și diversitatea datelor utilizate în procesul de antrenare. În contextul oftalmologiei computaționale, disponibilitatea bazelor de date publice a permis tranziția de la metodele clasice de procesare a imaginii la arhitecturi neuronale complexe, oferind un teren comun pentru evaluarea obiectivă a algoritmilor.

- **Seturi de referință pentru clasificarea globală (“Grading”)**

- **EyePACS** (Kaggle Diabetic Retinopathy Detection) [1], [7] rămâne cea mai vastă resursă publică disponibilă, conținând aproximativ 88.702 de imagini. Importanța sa strategică constă în eterogenitatea extremă a imaginilor (variații de expunere, focalizare, artefacte), fiind critic pentru antrenarea unor rețele backbone robuste capabile să gestioneze fenomenul de “domain shift” [8] și să generalizeze pe imagini de calitate slabă
- **APTOS 2019** Blindness Detection [7] reprezintă standardul modern de calitate. Colectat de Aravind Eye Hospital, acest set de 3.662 de imagini este utilizat frecvent în literatura recentă pentru “fine-tuning”, oferind o vizibilitate superioară a detaliilor retiniene comparativ cu seturile mai vechi
- **MESSIDOR-2** [7] este considerat în prezent „Benchmark-ul de Aur” pentru testarea algoritmilor. Extinzând setul original Messidor la 1.748 de imagini, acesta se distinge prin consistența diagnostică și este utilizat ca domeniu țintă (Target Domain) în studiile de adaptare [8] și pentru validarea sistemelor explicabile (XAI) [7], oferind o metrică fidelă a performanței reale pe date curate.
- **DeepDRiD** [15] aduce o inovație necesară prin structurarea celor 2.000 de imagini în perechi binoculare și includerea notelor de calitate, simulând fluxul de triaj clinic.
- **UoA-DR** [7] este o resursă istorică (200 imagini), utilizată complementar în studiile comparative pentru a asigura diversitatea demografică a validării.

- **Seturi specializate pentru segmentarea leziunilor și analiza detaliată**

- **IDRiD** [7] este setul critic pentru sarcinile de segmentare semantică fină [5]. Pentru 81 dintre imagini, experții au trasat manual măști binare la nivel de pixel, făcându-l indispensabil pentru antrenarea arhitecturilor complexe de tip Transformer și pentru validarea localizării leziunilor în sistemele end-to-end [7].
- **DDR** [17] a revoluționat segmentarea pe scară largă, oferind 13.673 de imagini. Este esențial pentru antrenarea rețelelor adânci de segmentare (precum RTNet) fără a risca “overfitting-ul”, oferind adnotări “pixel-wise” pentru leziuni multiple [5].

- **E-Ophtha** [7] este o bază de date vitală pentru reducerea alarmelor false în metodele de detecție a exudatelor [10]. Valoarea sa unică constă în includerea imaginilor cu patologii non-diabetice care pot fi confundate cu DR
- DiaRetDB1 [7] este standardul istoric pentru calibrarea sensibilității la leziuni roșii (microanevrisme) în lucrările de detecție automată [16]. Complementar, HEI-MED [14] oferă 169 de imagini focalizate exclusiv pe segmentarea zonelor de Edem Macular Diabetic.

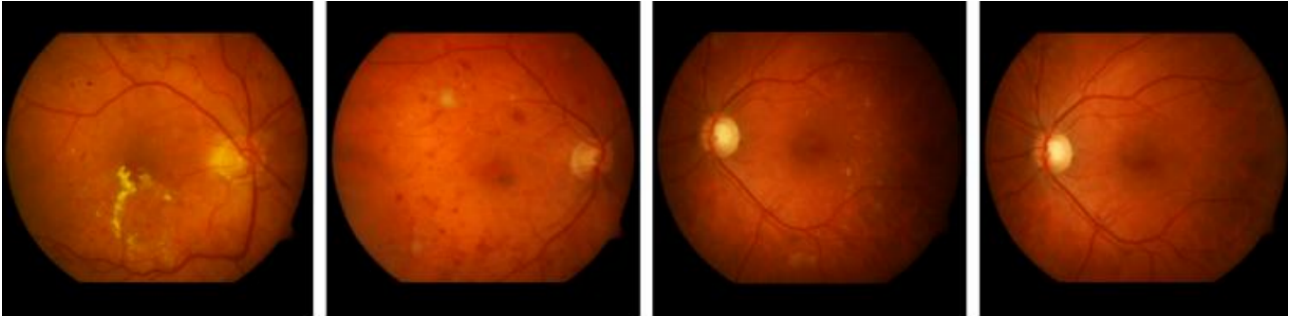


Figura 3.1: Exemple de imagini ale fundului de ochi în seturile de date DIARETDB0 și DIARETDB1 [7]

• Seturi standard pentru preprocesarea vasculară

- **DRIVE** [9] și **STARE** [7] sunt pilonii etapei de preprocesare. Deși conțin un număr redus de imagini, ele sunt utilizate obligatoriu pentru validarea algoritmilor de segmentare a vaselor de sânge, cum ar fi arhitecturile U-Net modificate [4] sau metodele clasice de extragere a trăsăturilor [16].

Tabelul de mai jos sintetizează caracteristicile tehnice esențiale ale seturilor de date analizate, evidențiind diferențele de volum, rezoluție și tipul de adnotare disponibil, elemente care dictează utilizarea lor în diversele etape ale proiectului.

Tabela 3.1: Tabel comparativ între seturile de date disponibile

Set de date	Număr imagini (total)	Rezoluție tipică (px)	Tip adnotare	Utilizare principală
EyePACS	88.702	Variabilă, până la 4000+	Globală, clase 0–4	Antrenare backbone și generalizare
DDR	13.673	Variabilă	Pixel-wise, măști și bounding box	Segmentare la scară largă
APTOS 2019	3.662	Înaltă, standardizată	Globală, clase 0–4	Fine-tuning și validare
DeepDRiD	2.000	512×512, reconstruit	Globală, calitate și binocular	Analiza calității imaginii
MESSIDOR-2	1.748	1440×960 – 2240×1488	Globală și edem	Benchmark și testare
IDRiD	516	4288×2848	Pixel-wise pentru 81 imagini și globală	Segmentare semantică fină
E-Ophtha	381 EX / 381 MA	Variabilă, medie/mare	Contur leziuni EX/MA	Reducerea alarmelor false pozitive
UoA-DR	~200	Variabilă	Globală, clase 0–4	Validare încrucișată
HEI-MED	169	2196×1958	Măști edem	Detectie edem macular
DiaRetDB1	89	1500×1152	Coordonate punctuale	Calibrare microanevrisme
DRIVE	40	565×584	Măști vase binare	Preprocesare și segmentare vase de sânge
STARE	20	700×605	Măști vase binare	Preprocesare și segmentare vase de sânge

3.2 Metoda bazată pe arhitectura Inception-V3 [1]

Studiul realizat de Gulshan et al. [1] este considerat un reper major în literatura de specialitate, marcând una dintre primele validări clinice riguroase ale utilizării rețelelor neuronale convoluționale pentru screening-ul automat al retinopatiei diabetice. Lucrarea demonstrează că un sistem de deep learning poate atinge și chiar depăși performanța medie a medicilor oftalmologi în detectarea retinopatiei diabetice, contribuind decisiv la acceptarea inteligenței artificiale în aplicații medicale reale [1].

Arhitectura utilizată este Inception-v3, o rețea convoluțională profundă caracterizată prin utilizarea modulelor Inception, care combină în paralel convoluții de dimensiuni diferite (1×1, 3×3, 5×5) și operații de pooling. Această strategie permite captarea informațiilor la multiple scări spațiale și reduce costul computațional prin utilizarea convoluțiilor 1×1 pentru diminuarea dimensionalității. Datorită acestei structuri, modelul este capabil să învețe reprezentări vizuale complexe, esențiale pentru identificarea leziunilor subtile specifice retinopatiei diabetice.

Modelul a fost antrenat pe un set de date extins, alcătuit din peste 128.000 de imagini fundus color, etichetate independent de mai mulți medici oftalmologi certificați în 2015. Utilizarea consensului între experți a contribuit la reducerea subiectivității etichetelor și la creșterea fiabilității procesului de antrenare. Sarcina principală a rețelei a fost clasificarea imaginilor în funcție de prezența sau absența retinopatiei diabetice referabile, conform standardelor clinice acceptate internațional [1].

Preprocesarea imaginilor a inclus decuparea automată a regiunii retiniene, redimensionarea la o rezoluție compatibilă cu Inception-v3 și normalizarea intensității pixelilor. În plus, au

fost aplicate tehnici extensive de augmentare a datelor, precum rotații, translații și ajustări de luminozitate și contrast, pentru a îmbunătăți generalizarea și a reduce supraînvățarea [1] [2].

Evaluarea performanței a fost realizată pe seturile de date EyePACS-1 și Messidor-2 [7], unde modelul a obținut o arie sub curba ROC (AUC) de aproximativ 0.99 pentru detectarea retinopatiei diabetice referabile, cu valori ridicate de sensibilitate, comparabile cu performanța medicilor oftalmologi experimentați [1] [11].

3.3 “Transfer Learning” pentru diagnosticul retinopatiei diabetice [2]

Lucrarea propusă de Jabbar et al. [2] abordează problema diagnosticului automat al retinopatiei diabetice dintr-o perspectivă practică, concentrându-se pe utilizarea tehnicilor de “transfer learning” pentru a compensa limitările impuse de dimensiunea redusă a seturilor de date medicale etichetate. Studiul evidențiază faptul că modelele CNN pre-antrenate pe seturi de date generale de mari dimensiuni, precum ImageNet, pot fi adaptate eficient pentru analiza imaginilor fundus, obținând performanțe competitive cu un cost de antrenare semnificativ redus [2].

Autorii investighează mai multe arhitecturi consacrate de rețele neuronale convoluționale, printre care VGG16, VGG19, ResNet50, Inception-v3 și DenseNet, utilizate ca extractoare de caracteristici. Strategia de transfer learning constă în inițializarea rețelelor cu ponderi pre-antrenate, urmată de ajustarea straturilor superioare pentru sarcina specifică de clasificare a retinopatiei diabetice. În unele experimente, straturile convoluționale inferioare sunt înghețate pentru a preveni supraînvățarea, în timp ce în altele este aplicat “fine-tuning” progresiv pentru a îmbunătăți adaptarea la domeniul imagisticii medicale. Una din abordările propuse poate fi observată în figura următoare.

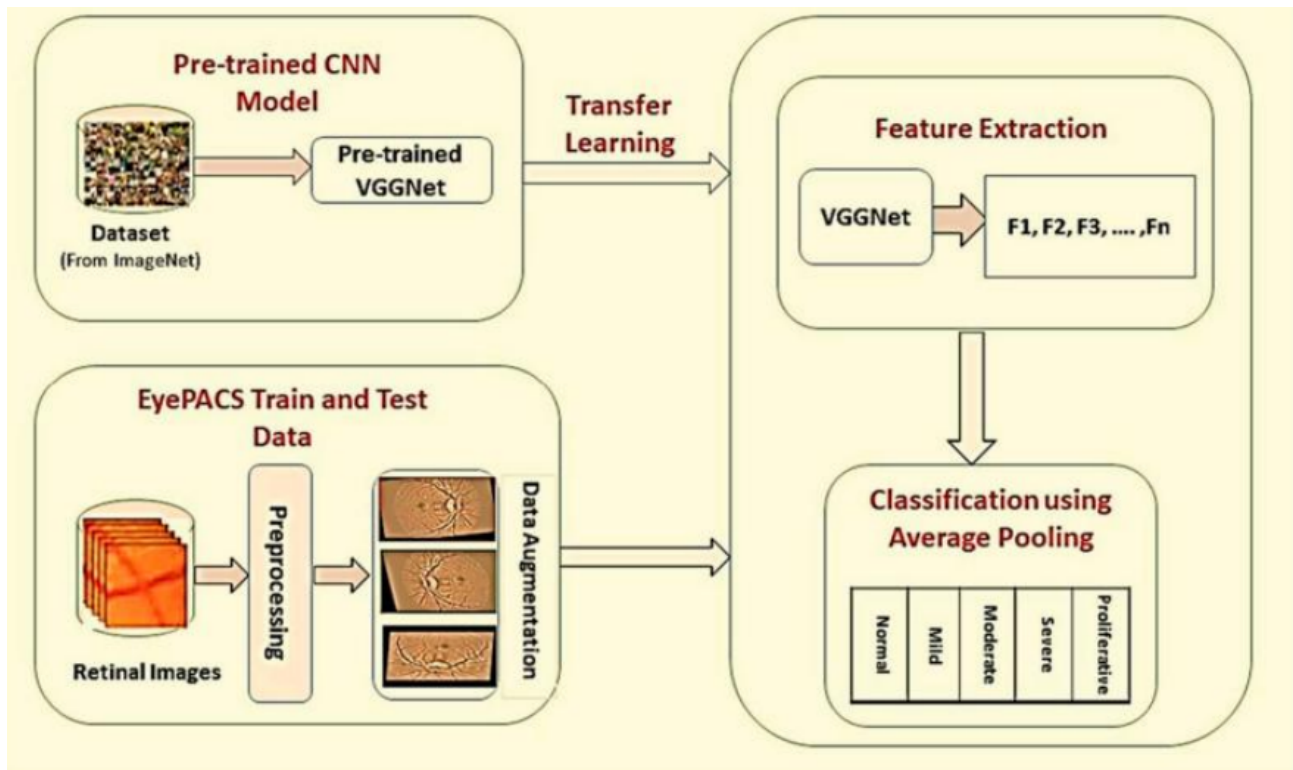


Figura 3.2: Framework propus pentru detectarea și clasificarea retinopatiei diabetice [2]

Seturile de date utilizate includ imagini fundus publice din Kaggle EyePACS [7], preprocesate prin redimensionare, normalizare și augmentare extensivă a datelor. Tehnicile de augmentare, precum rotațiile, flip-urile orizontale și ajustările de luminosități, joacă un rol central în creșterea diversității artificiale a datelor și în îmbunătățirea capacității de generalizare a modelelor. Această etapă este esențială în contextul “transfer learning-ului”, unde diferențele de distribuție dintre datele naturale și imaginile medicale pot afecta performanța rețelelor [2].

Evaluarea performanței este realizată folosind metrici standard, precum acuratețea, sensibilitatea, specificitatea și scorul F1, permițând o comparație riguroasă între diferitele arhitecturi analizate. Rezultatele experimentale indică faptul că modelele bazate pe DenseNet și ResNet obțin performanțe superioare față de arhitecturile mai vechi, confirmând avantajele reutilizării caracteristicilor vizuale generale în sarcini medicale specializate. Studiul subliniază că transfer learning-ul reprezintă o soluție viabilă și eficientă pentru implementarea rapidă a sistemelor automate de screening în contexte cu resurse limitate [2].

3.4 “Multi-task Learning” pentru gradarea retinopatiei diabetice [3]

Lucrarea propusă de Wang et al. [3] introduce o abordare de tip “deep multi-task learning” pentru gradarea automată a retinopatiei diabetice, având ca obiectiv principal exploatarea relației clinice dintre severitatea bolii și prezența leziunilor retiniene specifice. Spre deosebire de metodele clasice de clasificare “single-task”, autorii demonstrează că învățarea simultană a mai multor sarcini corelate clinic conduce la reprezentări vizuale mai discriminative și la performanțe superioare în sarcina principală de gradare [3].

Arhitectura propusă este alcătuită dintr-un backbone convoluțional comun, bazat pe o rețea CNN profundă (ResNet18), utilizat pentru extracția caracteristicilor globale din imaginile fundus. Peste acest extractor comun sunt construite mai multe ramuri specializate (task-specific branches), fiecare corespunzătoare unei sarcini distincte. Sarcina principală constă în clasificarea severității retinopatiei diabetice în mai multe stadii, în timp ce sarcinile auxiliare vizează detectarea prezenței diferitelor tipuri de leziuni, precum microanevrismele, exudatele dure, exudatele moi și hemoragiile, considerate “markeri” clinici relevanți ai progresiei bolii [3].

Un element central al arhitecturii îl reprezintă mecanismul de partajare controlată a caracteristicilor între sarcini. În locul unei separări rigide între ramuri, modelul permite propagarea informației relevante între sarcina de gradare și cele de detecție a leziunilor, încurajând învățarea unor reprezentări comune sensibile la semnalele patologice fine. Funcția de pierdere globală este definită ca o combinație ponderată a pierderilor individuale asociate fiecărei sarcini:

$$L_{isr} = \lambda_{img} \cdot L_{img_isr} + \lambda_{tv} \cdot L_{tv_isr} + \lambda_{sa} \cdot L_{seg-aware} + \lambda_{ca} \cdot L_{cls-aware}$$

,unde λ_{img} , λ_{tv} , λ_{sa} și λ_{ca} sunt hiperparametri care controlează contribuția pierderilor “intra-task”, “segmentation-aware” și “classification-aware”. Această formulare permite echilibrarea influenței fiecărei sarcini și stabilizează procesul de optimizare [3].

Modelul este antrenat și evaluat pe seturi de date publice de imagini fundus etichetate (DDR [17] și EyePACS [7]) atât cu stadiul retinopatiei diabetice, cât și cu adnotări la nivel de imagine privind prezența leziunilor. Imaginile sunt supuse unor etape standard de preprocesare, incluzând decuparea regiunii retiniene, redimensionarea și augmentarea datelor, pentru a reduce variațiile de iluminare și pentru a îmbunătăți capacitatea de generalizare. Prin includerea sarcinilor auxiliare, modelul beneficiază de un efect de regularizare implicită, reducând riscul

de supraînvăţare, aspect observat şi în lucrările anterioare bazate pe CNN-uri “single-task” [1], [11].

Rezultatele experimentale raportate în articol indică îmbunătăţiri consistente ale performanţei faţă de modelele “single-task”, atât în termeni de acurateţe, cât şi de scor F1 pentru sarcina de gradare a retinopatiei diabetice. În plus, autorii evidenţiază o creştere a stabilităţii predicţiilor şi o sensibilitate mai bună la stadiile intermediare ale bolii, considerate dificil de clasificat. Aceste rezultate confirmă faptul că integrarea informaţiilor despre leziuni în procesul de antrenare contribuie direct la o evaluare mai precisă a severităţii retinopatiei diabetice şi validează eficienţa paradigmei “multi-task learning” în context clinic [3].

3.5 Analiza arhitecturii “Modified U-Net” pentru segmentarea semantică [4]

Segmentarea semantică a structurilor retiniene, în special a reţelei vasculare şi a leziunilor asociate retinopatiei diabetice, reprezintă o etapă critică în diagnosticarea automată, deoarece modificările morfologice ale vaselor de sânge sunt adesea primii indicatori ai bolii. Deşi arhitectura UNet, introdusă iniţial de Ronneberger et al. [23] pentru segmentarea biomedicală, a devenit standardul de aur în domeniu datorită structurii sale simetrice de tip “encoder-decoder”, aceasta prezintă limitări în detectarea vaselor capilare foarte fine, pierzând informaţii spaţiale esenţiale în procesul de subeşantionare.

În studiul “Modified U-Net architecture for semantic segmentation of diabetic retinopathy images”, Sambyal et al [4]. propun o optimizare arhitecturală semnificativă pentru a contracara aceste deficienţe. Modelul propus, denumit Modified U-Net, păstrează structura în formă de „U” a reţelei originale, dar inovează fundamental la nivelul blocurilor de procesare.

Autorii au înlocuit blocurile convoluţionale standard cu blocuri reziduale (residual blocks), inspirate din arhitecturile de tip ResNet. Această modificare este crucială din punct de vedere tehnic: în reţelele adânci, apare frecvent problema dispariţiei gradientului (vanishing gradient), ceea ce face antrenarea dificilă şi duce la pierderea detaliilor fine. Prin introducerea conexiunilor reziduale (“skip connections” la nivel de bloc), informaţia originală este propagată mai eficient prin straturi, permiţând reţelei să înveţe caracteristici complexe fără a degrada performanţa pe detaliile de fineţe, precum microanevrismele sau vasele subţiri. Aceste detalii se pot observa şi în figura următoare.

Importanţa acestei abordări constă în capacitatea de generalizare comparativ cu metodele clasice de procesare a imaginii, precum cele bazate pe “clustering fuzzy” propuse anterior de Sopharak et al. [10]. În timp ce metodele de tip “clustering” sunt sensibile la zgomot şi la variaţiile de iluminare (frecvente în imaginile de fund de ochi), arhitectura Modified U-Net demonstrează o robusteţe sporită prin învăţarea contextuală a trăsăturilor

Pentru validarea performanţei, metoda a fost testată pe seturile de date publice standardizate DRIVE [9] şi STARE [7], recunoscute pentru dificultatea lor în segmentarea vaselor.

Rezultatele experimentale raportate de Sambyal et al. [4] au indicat o îmbunătăţire clară faţă de U-Net-ul standard. Pe setul de date DRIVE, autorii au obţinut o acurateţe maximă de 96.94 sub curba ROC (AUC) de 0.98, valori care plasează metoda în topul soluţiilor de segmentare. Mai mult, analiza vizuală a rezultatelor arată o continuitate mult mai bună a vaselor de sânge segmentate şi o reducere a alarmelor false în zonele cu patologii, demonstrând că integrarea învăţării reziduale în arhitectura de segmentare este o direcţie esenţială pentru analiza detaliată a retinopatiei diabetice.

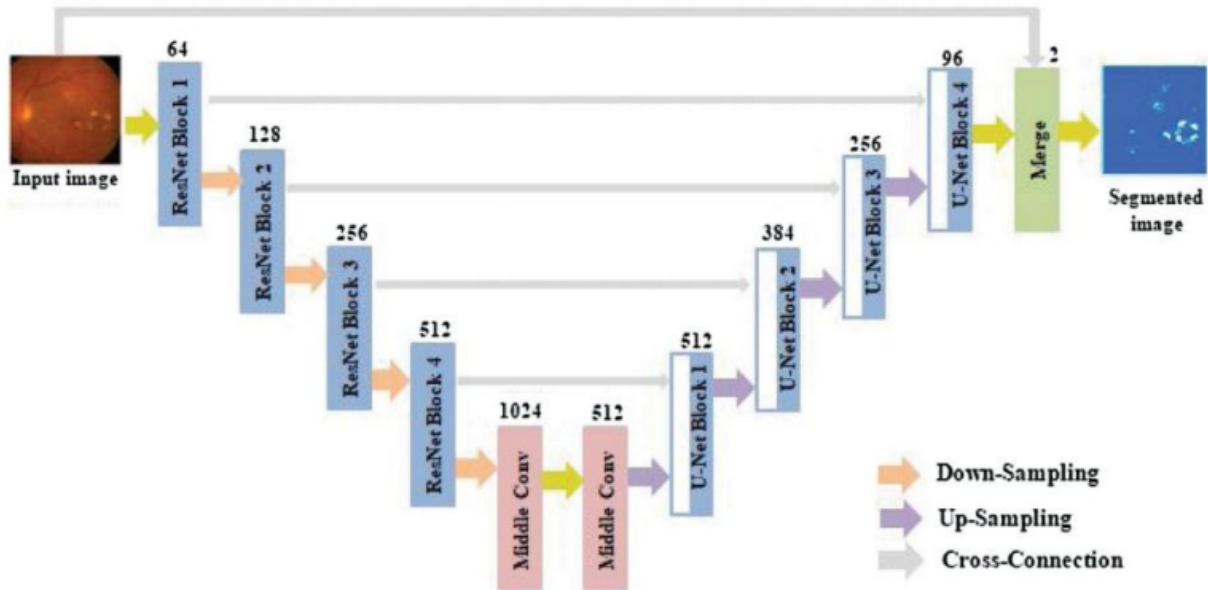


Figura 3.3: Arhitectura U-net modificată [4]

3.6 Tranziția către arhitecturi globale: RTNet (Relation Transformer Network) [5]

Dacă modelele bazate pe U-Net, precum cel discutat anterior [4], excelează în capturarea detaliilor locale, ele suferă totuși de o limitare inerentă a rețelelor convoluționale: incapacitatea de a modela dependențe la distanță mare (long-range dependencies). În patologia retiniană, leziunile nu apar izolat; există adesea o corelație spațială între poziția vaselor de sânge și apariția microanevrismelor sau a exudatelor. Pentru a adresa această lipsă de „context global”, Huang et al. [5] au propus în 2022 o schimbare de paradigmă prin introducerea RTNet (Relation Transformer Network), o arhitectură care marchează trecerea către mecanisme de atenție.

Inovația centrală a RTNet constă în înlocuirea parțială a operațiilor convoluționale standard cu blocuri de tip “Transformer”, denumite Relation Transformer Blocks (RTB). Spre deosebire de CNN-uri care „văd” doar o fereastră mică de pixeli la un moment dat, RTB-ul utilizează două mecanisme de atenție complementare:

1. “Self-Attention” (Atenție proprie): Permite modelului să înțeleagă relațiile dintre leziunile dispersate pe toată suprafața retinei (de exemplu, grupuri de exudate dure care semnalează o zonă de edem).
2. “Cross-Attention” (Atenție încrucișată): Autorii au observat că leziunile vasculare sunt strâns legate de structura vaselor de sânge. Astfel, mecanismul de “cross-attention” folosește harta vaselor de sânge ca un „ghid” pentru a localiza mai precis leziunile fine, transferând informație structurală dinspre ramura vasculară către ramura de detectare a leziunilor.

Pentru validare, metoda a fost testată pe seturile de date IDRiD [7] și DDR [17], care conțin adnotări la nivel de pixel pentru multiple tipuri de leziuni. Rezultatele cantitative au demonstrat că RTNet depășește metodele existente anterioare (inclusiv diverse variante de U-Net și DeepLab), obținând cele mai bune scoruri de segmentare pentru Exudate (EX), Microanevrisme (MA) și Exudate Moi (SE).

Deși inovația teoretică a arhitecturii RTNet (Relation Transformer Network) propusă de Huang et al. [5] este evidentă prin integrarea mecanismelor de atenție, validarea practică a acesteia necesită o demonstrație riguroasă a faptului că fiecare modul propus contribuie activ la îmbunătățirea segmentării. În acest sens, autorii au realizat un studiu extensiv de ablație (ablation study), analizând impactul incremental al componentelor arhitecturale asupra capacității de învățare și generalizare a rețelei.

Această analiză comparativă este sintetizată în graficele de performanță care monitorizează evoluția funcției de pierdere (Loss) și curbele Precision-Recall (PR) pentru cele patru tipuri principale de leziuni asociate retinopatiei diabetice: microanevrisme (MA), exudate (EX), hemoragii (HE) și exudate moi (SE).

Comparatia pornește de la un model “Baseline” (o rețea convoluțională standard de tip U-Net fără mecanisme de atenție) și adaugă progresiv componentele specifice RTNet, regasite în figura de mai jos:

1. GTB (Global Transformer Block): Adăugarea capacității de procesare globală.
2. SAH (Self-Attention Head): Introducerea atenției proprii pentru corelarea leziunilor similare.
3. RAH (Relation Attention Head / Cross-Attention): Modulul critic care leagă leziunile de structura vaselor de sânge.
4. RTB (Relation Transformer Block): Arhitectura completă propusă.

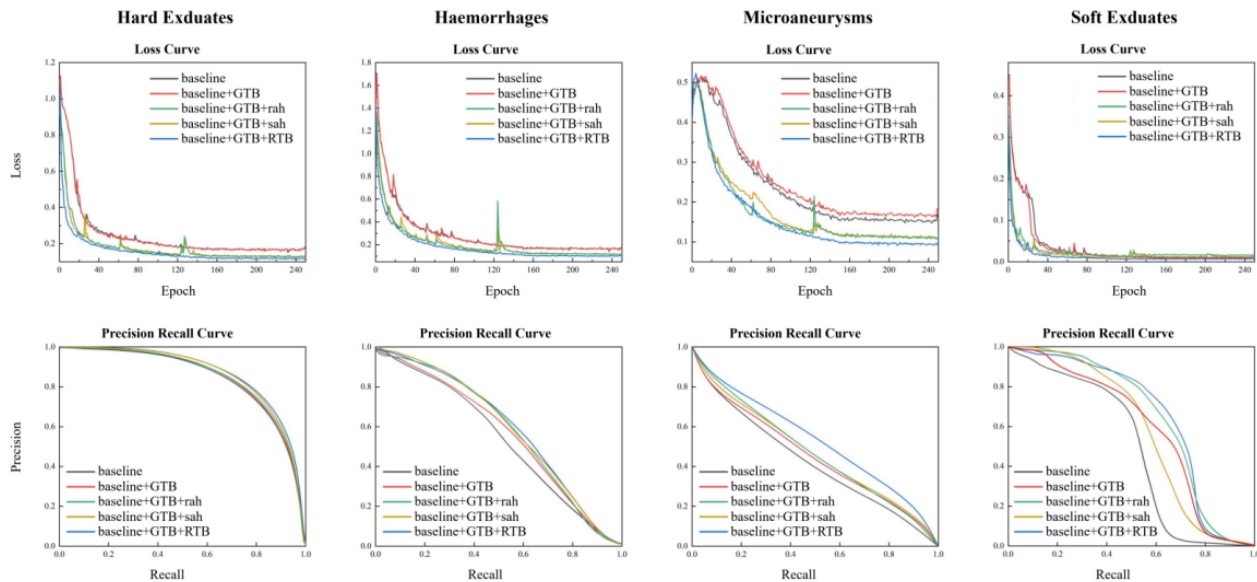


Figura 3.4: Curbele de Loss (stânga) și curbele Precision-Recall (dreapta) pentru studiile de ablație asupra celor patru leziuni DR. [5]

Lucrarea lui Huang et al. [5] validează ipoteza că integrarea informației globale prin Transformere este pasul necesar pentru a depăși plafonul de performanță atins de rețelele convoluționale pure în segmentarea medicală complexă.

3.7 Arhitectura “Dual Branch” [6]

Deși progresele în segmentarea semantică, ilustrate prin metodele bazate pe U-Net [4] sau pe Transformers [5], au permis o delimitare precisă a leziunilor, translatarea acestor detalii într-un diagnostic global (grading) rămâne o provocare majoră din cauza naturii subtile a diferențelor dintre stadiile retinopatiei diabetice. Rețelele convoluționale standard utilizate în studiile fundamentale, precum cel al lui Gulshan et al. [1], tind să piardă detaliile fine, cum ar fi microanevrismele incipiente, în procesul de redimensionare a imaginii necesar pentru analiza globală. Pentru a depăși acest compromis inerent între contextul vizual și rezoluția detaliilor, Shakibania et al. [6] au propus o arhitectură inovatoare de tip “Dual Branch Deep Learning Network”, care restructurează procesul de învățare prin decuplarea extragerii trăsăturilor în două fluxuri paralele distincte.

Conceptul central al acestei abordări constă în imitarea procesului cognitiv uman de diagnosticare, procesând imaginea simultan pe o „ramură globală” și o „ramură locală”. Ramura globală analizează imaginea completă a fundului de ochi pentru a captura structura generală a vaselor și geometria retinei, o strategie similară cu metodele clasice de “Transfer Learning” [2]. În paralel, ramura locală operează pe decupaje de înaltă rezoluție, fiind specializată în identificarea leziunilor microscopice care ar fi invizibile pentru rețeaua globală. Elementul de noutate adus de această lucrare față de abordările “multi-task” anterioare [3], este mecanismul de fuziune a celor două fluxuri, care utilizează tehnici de atenție pentru a pondera importanța trăsăturilor locale în decizia finală, asigurând că un detaliu critic nu este ignorat în favoarea unei imagini de ansamblu aparent sănătoase.

Eficiența acestei arhitecturi hibride este validată prin rezultatele experimentale obținute pe seturile de date standard, unde modelul “Dual Branch” a demonstrat o superioritate clară în clasificarea stadiilor intermediare ale bolii (ușor și moderat). Analiza performanței indică faptul că integrarea informației locale crește semnificativ sensibilitatea modelului și reduce confuzia între clasele adiacente, oferind o robustețe superioară la variațiile de calitate a imaginii comparativ cu arhitecturile “singlestream”. Astfel, lucrarea confirmă faptul că performanța actuală în 2024 nu derivă doar din adâncimea rețelei, ci din capacitatea arhitecturală de a reuni viziunea macroscopică cu cea microscopică într-un diagnostic coerent.

3.8 Abordarea “Explainable AI” [7]

În timp ce arhitecturile precum “Dual Branch Network” [6] sau Transformer-ele relaționale [5] au reușit să împingă limitele acurateței și ale segmentării fine, integrarea acestor sisteme în fluxul clinic real se lovește de o barieră fundamentală: lipsa de încredere a medicilor în algoritmi de tip „black-box”. Validarea clinică inițiată de Gulshan et al. [1] a demonstrat potențialul diagnostic, însă pentru ca un sistem de inteligență artificială să fie acceptat ca asistent de încredere, acesta trebuie să își poată justifica deciziile. În acest context, lucrarea publicată de Chetoui și Akhloufi [7], reprezintă un pilon esențial al stadiului actual al cunoașterii, adresând simultan două probleme critice: interpretabilitatea (Explainable AI - XAI) și robustețea pe seturi de date eterogene.

Metodologia propusă de autori se bazează pe o arhitectură de tip CNN modificată (utilizând backbone-uri precum DenseNet121 sau ResNet50), în care straturile clasice complet conectate sunt înlocuite cu Global Average Pooling (GAP). Această modificare tehnică permite generarea Hărților de Activare a Claselor (CAM), oferind medicului o confirmare vizuală a regiunilor patologice care au determinat diagnosticul.

Relevanța clinică a acestei abordări extensive este evidențiată în rezultatele vizuale, unde

hărțile de saliență generate de algoritm se suprapun cu precizie peste semnele patologice reale, indiferent de setul de date din care provine imaginea. Zonele marcate cu roșu intens pe hărțile CAM corespund anatomic cu microanevrismele și exudatele, confirmând că rețeaua neuronală a învățat trăsăturile intrinseci ale bolii și nu caracteristicile specifice ale camerei foto (bias-ul echipamentului). Prin urmare, modelul propus [7] validează faptul că un sistem trebuie să fie nu doar precis, ci și independent de sursa datelor și transparent în decizii.

3.9 Generalizarea în scenarii reale: “Multi-Model Domain Adaptation” [8]

Ultima frontieră în dezvoltarea sistemelor de diagnostic asistat de calculator (CAD) pentru retinopatia diabetică nu mai este legată strict de acuratețea obținută pe un singur set de date de antrenament, ci de capacitatea algoritmului de a funcționa corect într-un mediu clinic eterogen. În realitate, imaginile de fund de ochi variază drastic în funcție de modelul camerei, setările de iluminare, etnia pacientului sau protocolul de achiziție, fenomen cunoscut sub numele de “domain shift”. Modelele antrenate pe un „domeniu sursă” (de exemplu, imagini de înaltă calitate de la EyePACS [7]) tind să aibă performanțe scăzute atunci când sunt aplicate pe un „domeniu țintă” (imagini clinice noi, cu caracteristici vizuale diferite), ceea ce limitează sever aplicabilitatea lor universală.

Pentru a soluționa această problemă critică de implementare, Zhang et al. [8] au propus în 2022 o metodă inovatoare denumită “Multi-Model Domain Adaptation” (MMDA). Spre deosebire de abordările anterioare care încearcă să alinieze domeniile folosind un singur model, autorii introduc o arhitectură complexă bazată pe o rețea siameză, utilizând două modele distincte pentru a extrage trăsături complementare și pentru a crește robustețea deciziei.

Inovația tehnică centrală a lucrării constă în integrarea simultană a trei mecanisme de optimizare care lucrează concertat pentru a forța modelul să ignore diferențele stilistice dintre seturile de date și să se concentreze exclusiv pe patologie. Astfel, arhitectura combină o pierdere de clasificare pentru a asigura acuratețea diagnostică pe domeniul sursă etichetat, cu o pierdere de discrepanță care maximizează diferența dintre predicțiile celor două clasificatoare pe domeniul țintă pentru a detecta exemplele aflate la granița de decizie. În plus, se utilizează o pierdere de adaptare adversarială care, prin intermediul unui discriminator, minimizează distanța dintre distribuția trăsăturilor din sursă și cea din țintă, făcând ca imaginile provenite din spitale diferite să fie reprezentate similar de către rețeaua neuronală.

Validarea experimentală a metodei a fost realizată folosind setul de date EyePACS [7] ca domeniu sursă și Messidor-2 [7] ca domeniu țintă, simulând un scenariu realist de transfer tehnologic de la imagini de screening la imagini clinice de referință. Rezultatele au demonstrat că abordarea MMDA depășește semnificativ metodele standard de antrenare și alte tehnici de adaptare cunoscute, obținând o îmbunătățire substanțială a metricilor de performanță pe domeniul țintă fără a necesita etichete pentru acesta. Această lucrare încheie logic analiza stadiului actual al cunoașterii, subliniind faptul că un sistem complet nu trebuie doar să segmenteze leziuni fine sau să fie explicabil, ci trebuie să fie capabil să iasă din mediul controlat al laboratorului și să ofere o performanță robustă și generalizabilă în diversele condiții întâlnite în practica medicală globală.

3.10 Concluzii

Analiza literaturii de specialitate relevă faptul că detectarea automată a retinopatiei diabetice a depășit stadiul de experiment teoretic, devenind o direcție de cercetare matură, cu rezultate clinice valide. Totuși, problema rămâne una de o complexitate ridicată, care depășește simpla clasificare a imaginilor. Dificultatea majoră nu constă doar în volumul masiv de date necesar, ci în subtilitatea semnelor patologice incipiente, precum microanevrismele, care pot ocupa doar câțiva pixeli dintr-o imagine de înaltă rezoluție, fiind ușor de confundat cu artefactele sau zgomotul de fond. Această granularitate fină impune algoritmilor o capacitate de discriminare vizuală care, în anumite scenarii limită, trebuie să depășească acuitatea ochiului uman.

În ciuda acestor obstacole, soluțiile algoritmice existente demonstrează o evoluție remarcabilă. Trecerea de la rețelele convoluționale standard la arhitecturi sofisticate, capabile să integreze informația globală cu cea locală prin mecanisme de tip “dual-branch” [6], a permis o creștere semnificativă a sensibilității în detectarea stadiilor intermediare ale bolii. Mai mult, rezultatele obținute prin tehnici de adaptare la domeniu [8] sugerează că bariera variabilității echipamentelor medicale începe să fie depășită, deschizând calea pentru sisteme robuste, capabile să funcționeze corect indiferent de spitalul în care sunt implementate.

Un alt aspect promițător este schimbarea de paradigmă de la modelele de tip „cutie neagră” la sistemele explicabile (Explainable AI). Așa cum evidențiază studiile recente [7], capacitatea modelelor de a genera hărți de activare precise transformă inteligența artificială dintr-un simplu clasificator statistic într-un partener transparent pentru medic, facilitând validarea rapidă a deciziei. Deși provocările legate de dezechilibrul claselor și de generalizarea pe populații diverse persistă, performanțele atinse de metodele actuale, validate pe seturi de date riguroase precum MESSIDOR-2 [7], confirmă potențialul acestor tehnologii de a eficientiza screening-ul în masă și de a preveni orbirea la scară globală.

Capitolul 4

Setul de date și preprocesarea imaginilor

4.1 Descrierea setului de date utilizat

Pentru experimentele realizate în această lucrare a fost utilizat setul de date *Diabetic_Balanced_Data*, disponibil pe platforma Kaggle sub denumirea *Diabetic Retinopathy Balanced* [24]. Acesta conține imagini color ale fundului de ochi, organizate pentru problema de clasificare multclasă a retinopatiei diabetice. Spre deosebire de seturile de date puternic dezechilibrate, în care clasa fără retinopatie domină numeric, acest set de date este construit pentru a oferi o distribuție aproape uniformă între cele cinci clase de severitate.

În proiect, setul de date este stocat local în directorul:

```
datasets/Diabetic_Balanced_Data
```

Structura sa este compatibilă cu un flux standard de antrenare în PyTorch, fiind împărțită direct în trei subseturi:

- **train** – imaginile utilizate pentru actualizarea parametrilor modelului;
- **val** – imaginile utilizate pentru validarea modelului în timpul antrenării;
- **test** – imaginile păstrate pentru evaluarea finală.

Fiecare subset conține cinci directoare numerotate de la 0 la 4, fiecare director corespunzând unei clase de severitate:

- **0** – fără retinopatie diabetică;
- **1** – retinopatie diabetică ușoară;
- **2** – retinopatie diabetică moderată;
- **3** – retinopatie diabetică severă;
- **4** – retinopatie diabetică proliferativă.

Această organizare are un avantaj practic important: eticheta fiecărei imagini poate fi extrasă direct din numele folderului în care se află. Din acest motiv, clasa `BalancedDRDataset`, definită în fișierul `my_dataset.py`, parcurge automat directoarele 0, 1, 2, 3 și 4, salvează calea fiecărei imagini și atașează eticheta numerică corespunzătoare.

Setul este format din fișiere imagine în formate uzuale, precum `.jpg`, `.jpeg` și `.png`. În eșantionul verificat local, imaginile sunt stocate la rezoluția de 512×512 pixeli, însă pipeline-ul de antrenare nu depinde de această rezoluție inițială, deoarece toate imaginile sunt redimensionate la dimensiunea configurată prin parametrul `img_size`. În experimentele din această lucrare, valoarea implicită folosită este 224×224 pixeli.

Din punct de vedere metodologic, alegerea unui set de date echilibrat este importantă deoarece reduce tendința modelului de a favoriza clasa dominantă. În seturile medicale reale, dezechilibrul este frecvent, mai ales între clasele normale și clasele severe. Totuși, pentru compararea arhitecturilor și a funcțiilor de pierdere, un set echilibrat oferă un cadru experimental mai controlat, în care diferențele de performanță reflectă într-o măsură mai mare capacitatea modelului de a discrimina între clase.

4.2 Distribuția claselor

Distribuția imaginilor a fost calculată direct din structura locală a directoarelor. Rezultatul confirmă faptul că dataset-ul este aproape echilibrat pe cele cinci clase, cu o ușoară diferență pentru clasa 1. Tabelul 4.1 prezintă numărul de imagini pentru fiecare clasă și pentru fiecare subset.

Tabela 4.1: Distribuția imaginilor în setul de date *Diabetic_Balanced_Data*

Clasa	Train	Validare	Test	Total
0	7000	2000	1000	10000
1	6792	1940	971	9703
2	7000	2000	1000	10000
3	7000	2000	1000	10000
4	7000	2000	1000	10000
Total	34792	9940	4971	49703

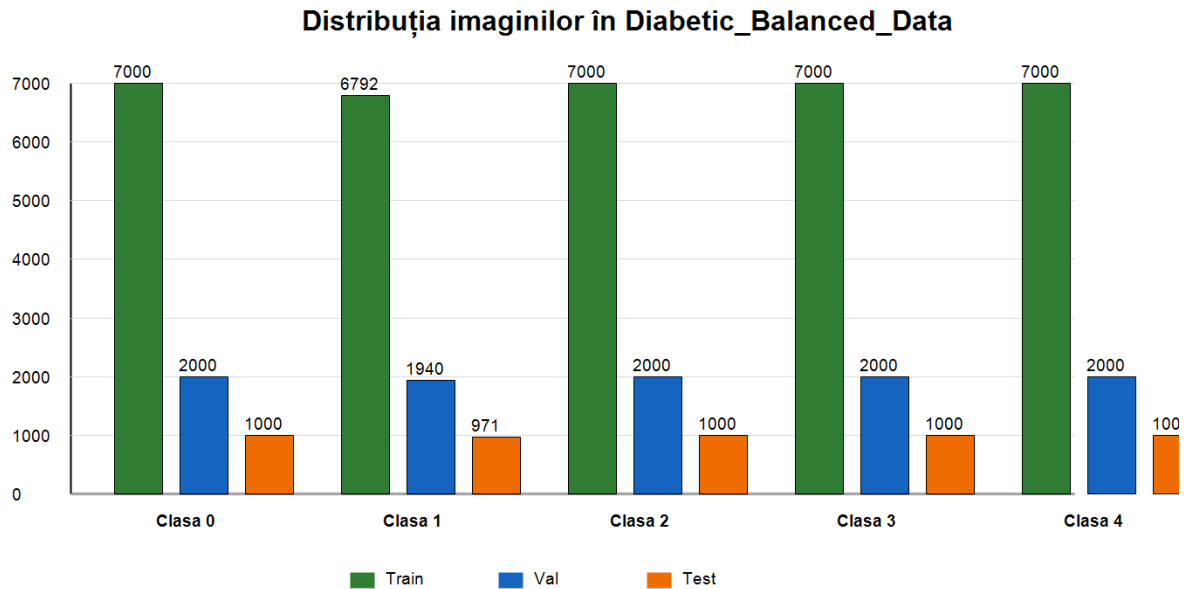


Figura 4.1: Distribuția imaginilor pe clase și subseturi în *Diabetic_Balanced_Data*

Se observă că fiecare dintre clasele 0, 2, 3 și 4 conține exact 10000 de imagini în total, împărțite în 7000 pentru antrenare, 2000 pentru validare și 1000 pentru testare. Clasa 1 conține 9703 imagini, ceea ce reprezintă o diferență relativ mică față de celelalte clase. Această abatere nu modifică semnificativ caracterul echilibrat al setului, dar este utilă în interpretarea rezultatelor, mai ales dacă modelul prezintă performanțe ușor diferite pe clasa retinopatiei ușoare.

Distribuția aproape uniformă are mai multe avantaje:

- reduce riscul ca modelul să învețe o strategie trivială de favorizare a clasei majoritare;
- permite compararea mai clară a performanței pe fiecare clasă;
- face matricea de confuzie mai ușor de interpretat;
- ajută la evaluarea claselor severe, care în seturile reale sunt de obicei mai puțin reprezentate;
- susține experimente comparative între funcții de pierdere, precum Cross-Entropy, Weighted Cross-Entropy și Focal Loss.

Totuși, distribuția echilibrată trebuie interpretată cu atenție. Într-un scenariu clinic real, prevalența claselor nu este neapărat uniformă, iar un sistem implementat în practică ar trebui validat și pe date cu distribuție naturală. În această lucrare, setul echilibrat este utilizat pentru a evalua capacitatea modelelor de a separa vizual cele cinci clase, nu pentru a estima direct prevalența clinică a fiecărui stadiu.

4.3 Probleme specifice imaginilor medicale

Imaginile de fund de ochi ridică o serie de probleme specifice, discutate teoretic în capitolul anterior și întâlnite practic și în setul utilizat. Chiar dacă imaginile sunt deja organizate și echilibrate, procesul de clasificare rămâne dificil din cauza variabilității vizuale și a caracterului subtil al leziunilor.

Principalele dificultăți asociate imaginilor din acest set sunt:

- **Diferențe de iluminare și contrast.** Unele imagini pot avea zone mai întunecate sau mai luminoase, ceea ce influențează vizibilitatea leziunilor.
- **Variații cromatice.** Tonul fundului de ochi poate diferi între pacienți și între condițiile de achiziție.
- **Leziuni de dimensiuni reduse.** Microanevrismele sau hemoragiile punctiforme pot ocupa regiuni foarte mici din imagine.
- **Asemănarea dintre clase adiacente.** Clasele 0, 1 și 2 sunt mai greu de separat decât clasele avansate, deoarece progresia bolii este graduală.
- **Artefacte și variații de calitate.** Reflexiile, blur-ul sau marginile întunecate pot influența atât examinarea umană, cât și predicția automată.

În plus, un aspect particular al setului local este prezența unor imagini augmentate direct în structura dataset-ului, identificabile prin secvența `_aug_` în numele fișierului. Distribuția locală indică 23620 de imagini augmentate în subsetul de antrenare, 6766 în validare și 3365 în test. Acest lucru sugerează că dataset-ul Kaggle conține deja o componentă de echilibrare/augmentare înainte de augmentările aplicate dinamic în timpul antrenării.

Această particularitate are două implicații importante:

- imaginile augmentate contribuie la echilibrarea numerică a claselor și la creșterea diversității vizuale;

- augmentările aplicate suplimentar în timpul antrenării trebuie alese moderat, pentru a evita deformarea excesivă a semnelor patologice.

În fluxul de lucru al acestei lucrări, augmentările dinamice sunt aplicate doar pentru subsetul de antrenare, nu și pentru validare sau testare. Această separare este importantă deoarece validarea și testarea trebuie să reflecte performanța modelului pe imagini stabile, fără variații aleatorii introduse la fiecare epocă.

4.4 Etapele de preprocesare

Preprocesarea imaginilor este realizată în clasa `BalancedDRDataset`, care extinde clasa `Dataset` din PyTorch. Rolul acestei clase este de a încărca imaginea din directorul corespunzător, de a converti imaginea în format RGB, de a-i atașa eticheta numerică și de a aplica transformările necesare înainte ca imaginea să fie transmisă către model.

Fluxul de încărcare a datelor poate fi rezumat astfel:

1. se selectează subsetul dorit: `train`, `val` sau `test`;
2. se parcurg directoarele claselor 0, 1, 2, 3, 4;
3. se colectează imaginile cu extensiile `.png`, `.jpg` și `.jpeg`;
4. eticheta fiecărei imagini este preluată din numele directorului clasei;
5. imaginea este deschisă cu biblioteca PIL și convertită în RGB;
6. se aplică transformările specifice subsetului;
7. imaginea și eticheta sunt returnate ca tensor pentru antrenare sau evaluare.

Transformările comune pentru toate subseturile sunt:

- **redimensionarea** la 224×224 pixeli, prin parametrul `image_size`;
- **conversia în tensor**, necesară pentru procesarea cu PyTorch;
- **normalizarea** folosind mediile și deviațiile standard ImageNet:

$$\mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225]$$

Normalizarea cu valorile ImageNet este justificată prin faptul că modelele utilizate în această lucrare, precum ResNet50 și InceptionV3, sunt inițializate cu ponderi pre-antrenate. Astfel, imaginile de intrare sunt aduse într-un interval statistic compatibil cu datele pe care aceste arhitecturi au fost antrenate inițial [20], [21].

În implementare există și o tratare defensivă a erorilor de citire: dacă o imagine nu poate fi deschisă, se creează o imagine RGB neagră de dimensiunea dorită. Această soluție previne oprirea antrenării din cauza unui fișier corupt, însă în practică astfel de cazuri ar trebui investigate și eliminate din setul final de date.

4.5 Augmentarea datelor

Augmentarea datelor este folosită pentru a crește robustețea modelului la variații de poziție, orientare și iluminare. În cazul imaginilor retiniene, augmentările trebuie alese cu atenție: ele trebuie să simuleze variații realiste ale achiziției, fără să modifice semnificația clinică a imaginii.

În clasa `BalancedDRDataset`, augmentarea este aplicată doar pentru subsetul `train`. Transformările folosite sunt:

- **RandomHorizontalFlip** cu probabilitatea 0.5;
- **RandomVerticalFlip** cu probabilitatea 0.5;
- **RandomRotation** cu un unghi maxim de 15 grade;
- **ColorJitter** pentru variații moderate de luminozitate și contrast;
- redimensionare, conversie în tensor și normalizare.

Flip-urile orizontale și verticale pot fi acceptabile în acest context deoarece diagnosticul global al retinopatiei diabetice depinde de prezența și distribuția leziunilor, nu de orientarea exactă stânga-dreapta a imaginii. Rotațiile moderate ajută modelul să fie mai puțin sensibil la mici diferențe de aliniere, iar modificările reduse de luminozitate și contrast simulează variații între camere sau condiții de captură.

Pentru subseturile `val` și `test`, augmentarea aleatorie nu este aplicată. În aceste faze, transformările sunt limitate la:

- redimensionarea imaginii;
- conversia în tensor;
- normalizarea cu valorile ImageNet.

Această diferență între antrenare și evaluare este importantă. Dacă validarea sau testarea ar include augmentări aleatorii, performanța modelului ar putea varia între rulări, iar comparația dintre experimente ar deveni mai puțin stabilă. Prin păstrarea unui pipeline determinist pentru validare și testare, metricile obținute reflectă mai clar comportamentul modelului.

Trebuie menționat și faptul că dataset-ul conține deja imagini augmentate în folderele sale. Prin urmare, augmentarea dinamică aplicată la antrenare nu este singura sursă de diversitate. Această combinație ajută la creșterea variației vizuale, dar poate introduce și imagini foarte apropiate între ele. Din acest motiv, performanța finală trebuie analizată nu doar prin acuratețe, ci și prin matrice de confuzie, scor AUC și comportamentul pe fiecare clasă.

4.6 Împărțirea datelor în seturi de antrenare, validare și testare

Împărțirea datelor este deja furnizată de structura dataset-ului. În loc să se realizeze o împărțire manuală în cod, experimentele folosesc direct directoarele `train`, `val` și `test`. Această abordare reduce riscul de erori în pregătirea datelor și simplifică reproducerea experimentelor.

Rolul fiecărui subset este următorul:

- **Setul de antrenare** este folosit pentru actualizarea ponderilor modelului prin backpropagation.
- **Setul de validare** este folosit pentru monitorizarea performanței în timpul antrenării și pentru alegerea celui mai bun model.
- **Setul de testare** este folosit doar la final, pentru evaluarea obiectivă a modelului salvat.

În scriptul de antrenare, aceste subseturi sunt încărcate prin trei instanțe separate ale clasei `BalancedDRDataset`:

- `BalancedDRDataset(root_dir, split='train')`
- `BalancedDRDataset(root_dir, split='val')`
- `BalancedDRDataset(root_dir, split='test')`

Ulterior, fiecare dataset este transmis către un `DataLoader`. Pentru subsetul de antrenare se folosește `shuffle=True`, astfel încât ordinea imaginilor să fie amestecată la fiecare epocă. Pentru validare și testare se folosește `shuffle=False`, deoarece în aceste faze nu este necesară randomizarea ordinii.

Raportul aproximativ al împărțirii este 70% pentru antrenare, 20% pentru validare și 10% pentru testare. Această proporție este potrivită pentru experimentele din lucrare deoarece oferă un set de antrenare suficient de mare, păstrând în același timp un număr consistent de imagini pentru validare și testare.

Prin această organizare, capitolul experimental pornește de la un set de date clar structurat, aproape echilibrat și compatibil cu evaluarea multiclasă. Particularitățile dataset-ului, precum existența imaginilor augmentate și distribuția aproape uniformă a claselor, trebuie avute în vedere în interpretarea rezultatelor obținute în capitolele următoare.

Capitolul 5

Metodologia propusă

5.1 Fluxul general al sistemului

Metodologia propusă urmărește construirea unui flux complet pentru clasificarea automată a retinopatiei diabetice pe baza imaginilor de fund de ochi. Sistemul pornește de la setul de date *Diabetic_Balanced_Data*, organizat în directoare de antrenare, validare și testare, și ajunge la salvarea unor modele antrenate care pot fi folosite ulterior într-o aplicație de inferență. Implementarea este realizată în Python, folosind PyTorch pentru antrenarea rețelelor, Torchvision și `timm` pentru arhitecturi pre-antrenate, iar scikit-learn, Matplotlib și Seaborn pentru evaluare și vizualizare.

Fluxul de lucru este împărțit în trei componente principale. Fișierul `my_dataset.py` se ocupă de încărcarea imaginilor și de aplicarea transformărilor. Fișierul `builder.py` definește modelele, funcțiile de pierdere, modul de calcul al predicțiilor și optimizatorii. Scriptul `train.py` integrează aceste componente, rulează antrenarea, validează modelul la fiecare epocă, salvează cea mai bună variantă și generează graficele de evaluare.

Problema este formulată ca o clasificare multiclasă în cinci clase, corespunzătoare stadiilor retinopatiei diabetice. Pentru fiecare imagine, modelul produce un vector de cinci scoruri, iar clasa finală este obținută prin alegerea valorii maxime în cazul funcțiilor de pierdere de tip Cross-Entropy și Focal Loss. Pentru varianta ordinală, ieșirea este interpretată prin praguri binare, deoarece clasele au o ordine clinică naturală.

5.2 Modelele analizate

În experimente au fost analizate arhitecturi convoluționale moderne, alese deoarece permit transfer learning și au fost utilizate frecvent în clasificarea imaginilor. Implementarea permite construirea modelelor `resnet50`, `efficientnet_b3`, `inception_v3` și `incres_v2`. Accentul lucrării este pus pe ResNet50, InceptionV3 și Inception-ResNet-V2, deoarece acestea oferă trei perspective diferite asupra problemei: învățare reziduală, procesare multi-scală și combinația dintre cele două.

Toate modelele sunt adaptate pentru cinci clase. Straturile finale, proiectate inițial pentru clasificarea ImageNet, sunt înlocuite sau configurate astfel încât ieșirea să corespundă celor cinci stadii ale retinopatiei diabetice. Folosirea ponderilor pre-antrenate este importantă deoarece permite reutilizarea unor trăsături vizuale generale, precum muchii, texturi și forme, care sunt apoi ajustate pentru domeniul medical.

5.3 Arhitectura ResNet50

ResNet50 este o arhitectură convoluțională bazată pe conexiuni reziduale [20]. Ideea principală este ca un bloc al rețelei să nu învețe direct o transformare completă, ci o corecție față

de intrarea primită. Această abordare facilitează antrenarea rețelelor adânci și reduce riscul degradării performanței odată cu creșterea numărului de straturi.

În contextul imaginilor de fund de ochi, ResNet50 poate extrage trăsături ierarhice: primele straturi detectează contururi, variații de culoare și texturi simple, iar straturile profunde combină aceste informații în structuri mai complexe, precum vase retiniene, zone de exudate, hemoragii sau regiuni suspecte. În implementarea din `builder.py`, stratul final `fc` este înlocuit cu o secvență formată din `Dropout(0.5)` și un strat liniar cu cinci ieșiri. Dropout-ul are rol de regularizare și reduce tendința de supraînvățare.

5.4 Arhitectura InceptionV3

InceptionV3 este o arhitectură construită în jurul modulelor Inception, care procesează aceeași intrare prin filtre convoluționale de dimensiuni diferite [21]. Această strategie permite extragerea simultană a trăsăturilor locale și a celor cu context spațial mai larg. Pentru retinopatia diabetică, acest lucru este relevant deoarece semnele patologice pot avea dimensiuni foarte diferite: microanevrismele sunt mici și discrete, în timp ce hemoragiile sau exudatele pot ocupa regiuni mai mari.

În cod, InceptionV3 este încapsulat prin clasa `InceptionWrapper`. Această clasă rezolvă particularitatea modelului Torchvision, care în timpul antrenării poate returna și o ieșire auxiliară. Pentru a păstra aceeași interfață cu funcțiile de pierdere, wrapper-ul returnează doar predicția principală. Ultimul strat complet conectat este înlocuit cu un strat liniar cu cinci ieșiri, corespunzătoare claselor 0–4.

5.5 Arhitectura Inception-ResNet-V2

Inception-ResNet-V2 este o arhitectură hibridă care combină modulele Inception cu conexiuni reziduale. Din familia Inception preia ideea procesării multi-scală, prin ramuri convoluționale paralele, iar din familia ResNet preia conexiunile care facilitează propagarea gradientului. Rezultatul este o rețea profundă, capabilă să învețe reprezentări vizuale bogate fără a pierde stabilitatea antrenării.

Această arhitectură este potrivită pentru imaginile retiniene deoarece decizia de severitate poate depinde atât de leziuni mici, cât și de distribuția globală a modificărilor patologice. Ramurile Inception pot surprinde structuri la scări diferite, iar conexiunile reziduale ajută la păstrarea informației utile de-a lungul rețelei. În implementare, modelul este construit cu biblioteca `timm`, prin apelul `timm.create_model('inception_resnet_v2', pretrained=pretrained, num_classes=5)`. În comparație cu ResNet50, modelul are o capacitate mai mare, dar necesită și mai multe resurse de calcul.

5.6 Funcții de pierdere utilizate

Funcția de pierdere definește modul în care modelul este penalizat pentru predicții greșite. În această lucrare au fost folosite mai multe variante: Cross-Entropy, Weighted Cross-Entropy, Focal Loss și o variantă ordinală bazată pe `BCEWithLogitsLoss`. Cross-Entropy este alegerea standard pentru clasificare multclasă și oferă un punct de referință stabil.

Weighted Cross-Entropy permite introducerea unor ponderi pentru clase, fiind utilă atunci când unele clase trebuie penalizate mai puternic. Focal Loss, implementată prin clasa `FocalLossMultiClass`

pornește de la Cross-Entropy, dar reduce contribuția exemplor ușoare și acordă mai multă importanță celor dificile. Parametrul `gamma` este transmis din linia de comandă; în experimentele inițiale a fost analizat intervalul 1.5–2.0, iar valoarea 1.6 a fost selectată pentru experimentele extinse deoarece a oferit cel mai bun compromis între acuratețe, AUC și distribuția erorilor. Varianta ordinală exploatează faptul că stadiile retinopatiei diabetice au o ordine naturală, transformând eticheta într-un vector care descrie nivelurile de severitate depășite.

5.7 Optimizatorul și hiperparametrii de antrenare

Optimizarea parametrilor este realizată prin funcția `get_optimizer`, care permite selectarea între Adam, AdamW și SGD. Configurațiile principale folosesc rata de învățare 0.0001, iar pentru Adam și AdamW este aplicată și regularizarea prin `weight_decay`. Adam este potrivit pentru aceste experimente deoarece adaptează rata de actualizare pentru fiecare parametru și oferă o convergență stabilă în multe probleme de clasificare.

Hiperparametrii sunt transmiși prin linia de comandă în `train.py`. Printre cei mai importanți se află `model`, `loss_name`, `optimizer`, `epochs`, `batch_size`, `lr`, `img_size`, `num_workers`, `gamma`, `train_source`, `test_source` și `test_sources`. Scriptul folosește și un scheduler de tip `ReduceLROnPlateau` care reduce rata de învățare atunci când AUC-ul de validare nu se mai îmbunătățește.

5.8 Strategia de evaluare

Evaluarea se realizează în două etape. În timpul antrenării, modelul este monitorizat pe setul de validare, iar cea mai bună variantă este salvată pe disc. La final, modelul salvat este reîncărcat și evaluat pe setul de testare. Pentru experimentele de generalizare între domenii, implementarea permite evaluarea aceluiași model pe mai multe seturi de test în aceeași rulare, prin argumentul `test_sources`. Astfel, un model antrenat pe date combinate poate fi testat separat pe APTOS și pe setul Balanced, fără reluarea antrenării. Această separare este necesară pentru ca testarea să rămână o evaluare cât mai obiectivă a modelului ales.

Metricile urmărite sunt funcția de pierdere, acuratețea, AUC-ul multiclasă, raportul de clasificare, matricea de confuzie și curbele ROC. AUC-ul este calculat în regim one-vs-rest și mediat macro, astfel încât fiecare clasă să contribuie egal la scorul final. În plus, scriptul include generarea de hărți Grad-CAM, utile pentru a observa regiunile imaginii care influențează predicția modelului.

5.9 Implementarea procesului de antrenare

Scriptul `train.py` coordonează întregul proces experimental. După citirea argumentelor, acesta selectează dispozitivul de calcul disponibil, creează directoarele pentru rezultate și încarcă datele prin clasele de dataset definite în `my_dataset.py`. Implementarea permite folosirea surselor `balanced`, `balanced_aug`, `aptos`, precum și a combinațiilor `combined` și `balanced_aug_aptos`. Fiecare dataset este transmis către un `DataLoader`, cu `shuffle=True` pentru antrenare și `shuffle=False` pentru validare și testare.

Pentru fiecare epocă, modelul trece printr-o fază de antrenare și una de validare. În faza de antrenare se calculează gradientul și se actualizează parametrii, iar în faza de validare calculele sunt realizate fără gradient. Dacă metrica de selecție se îmbunătățește, ponderile sunt salvate într-un fișier `.pth`. La final, scriptul generează graficele de istoric, matricea de confuzie, curbele

ROC și, atunci când este posibil, imaginile Grad-CAM. În cazul evaluării pe mai multe surse de test, fiecare set primește fișiere separate, marcate în nume prin sufixe precum `test_aptos`, `test_balanced` sau `test_balanced_aug`.

Capitolul 6

Rezultate experimentale

6.1 Configurația experimentală

Experimentele au fost realizate pe setul de date *Diabetic_Balanced_Data*, format din 34792 imagini pentru antrenare, 9940 imagini pentru validare și 4971 imagini pentru testare. Imaginile au fost redimensionate la dimensiunea configurată prin parametrul `img_size`, normalizate cu valorile ImageNet și încărcate în batch-uri prin `DataLoader`. Pentru subsetul de antrenare au fost aplicate augmentări dinamice, iar pentru validare și testare au fost păstrate transformări deterministe.

Configurațiile salvate includ variante ResNet50 cu Cross-Entropy, Weighted Cross-Entropy, Focal Loss, Focal Loss cu AdamW și BCE ordinal, precum și variante InceptionV3 și Inception-ResNet-V2 cu Focal Loss. Modelele rezultate sunt salvate în directorul `Rezultate/modele`, iar graficele în `Rezultate/grafice`. Această organizare permite compararea directă între arhitecturi și între funcții de pierdere.

Pe lângă experimentele inițiale, au fost adăugate experimente extinse pentru analiza generalizării între seturi de date. Acestea folosesc antrenare pe seturi combinate și evaluare separată pe fiecare domeniu de testare. Astfel, în loc ca rezultatul final să fie calculat pe un singur set concatenat, același model este evaluat separat pe subsetul de test APTOS și pe subsetul de test Balanced, respectiv Balanced Aug. Această strategie permite observarea diferențelor de performanță între domenii și reduce riscul ca un scor global să ascundă performanțe slabe pe unul dintre seturi.

Tabela 6.1: Experimente extinse adăugate pentru evaluare pe seturi de test separate

Exp.	Model	Train/Val	Test separat	Acc.	AUC	F1 macro
7.1	ResNet50	Balanced + APTOS matched	APTOS test	–	–	–
7.1	ResNet50	Balanced + APTOS matched	Balanced test	–	–	–
7.2	ResNet50	Balanced Aug + APTOS matched	APTOS test	–	–	–
7.2	ResNet50	Balanced Aug + APTOS matched	Balanced Aug test	–	–	–
8.1	Inception-ResNet-V2	Balanced + APTOS matched	APTOS test	–	–	–
8.1	Inception-ResNet-V2	Balanced + APTOS matched	Balanced test	–	–	–
8.2	Inception-ResNet-V2	Balanced Aug + APTOS matched	APTOS test	–	–	–
8.2	Inception-ResNet-V2	Balanced Aug + APTOS matched	Balanced Aug test	–	–	–

În tabel, Balanced reprezintă setul *Diabetic Balanced Data*, Balanced Aug reprezintă varianta augmentată și preprocesată Ben Graham, iar APTOS matched reprezintă subsetul APTOS preprocesat prin aceeași transformare. Toate experimentele extinse folosesc Focal Loss cu $\gamma=1.6$, optimizator Adam, rata de învățare 0.0001 și batch size 32. Numărul de epoci este mărit față de experimentele inițiale, pentru a observa dacă modelele continuă să învețe după primele 15 epoci. Valorile marcate cu “—” reprezintă rezultate care urmează să fie completate după finalizarea antrenărilor.

6.2 Evoluția funcției de pierdere

Evoluția funcției de pierdere indică stabilitatea procesului de învățare. În mod ideal, pierderea de antrenare scade treptat, iar pierderea de validare urmează o tendință apropiată, fără diferențe mari între cele două. O scădere doar pe antrenare, însoțită de stagnare sau creștere pe validare, ar indica supraînvățare.

În cazul modelelor antrenate prin transfer learning, scăderea pierderii arată că straturile finale și reprezentările interne se adaptează la imaginile retiniene. Pentru Focal Loss, valoarea numerică trebuie interpretată separat de Cross-Entropy, deoarece această funcție modifică penalizarea exemplilor ușoare și pune accent pe exemplele dificile. Din acest motiv, analiza pierderii este completată prin acuratețe, AUC și matricea de confuzie.

6.3 Evoluția acurateții pe setul de validare

Acuratețea pe setul de validare măsoară proporția imaginilor clasificate corect în timpul antrenării. Deoarece dataset-ul este aproape echilibrat, această metrică este mai relevantă decât într-un scenariu puternic dezechilibrat. Totuși, acuratețea globală nu arată ce clase sunt clasificate corect și ce clase sunt confundate.

Curbele de acuratețe permit observarea configurațiilor stabile și a celor care oscilează. Un model care crește constant pe validare indică o adaptare bună la date, în timp ce oscilațiile mari pot sugera o rată de învățare prea mare, un număr redus de epoci sau sensibilitate la exemplele dificile. Pentru o problemă medicală, această metrică trebuie interpretată împreună cu performanța pe clase, în special pentru stadiile severe.

6.4 Analiza scorului AUC

Scorul AUC este calculat în regim multiclăsă one-vs-rest și mediat macro. Această metrică evaluează capacitatea modelului de a separa fiecare clasă de restul claselor, folosind probabilitățile produse de softmax. Spre deosebire de acuratețe, AUC-ul ia în considerare ordonarea scorurilor și este mai potrivit pentru analiza separabilității.

În scriptul de antrenare, AUC-ul de validare este folosit ca metrică principală pentru salvarea modelului în majoritatea configurațiilor. Această alegere este justificată deoarece o predicție medicală nu trebuie analizată doar ca etichetă finală, ci și prin distribuția probabilităților. Un model cu AUC ridicat poate separa bine clasele chiar și atunci când unele predicții finale rămân ambigue.

6.5 Analiza curbelor ROC multclasă

Curbele ROC multclasă sunt generate pentru fiecare clasă separat, în regim one-vs-rest. Pentru fiecare clasă, curba arată relația dintre rata adevărat pozitivă și rata fals pozitivă la praguri diferite. Astfel, se poate observa dacă modelul separă mai bine clasele extreme sau dacă are dificultăți în clasele intermediare.

În retinopatia diabetică, clasele 0 și 4 pot fi mai ușor de distins, deoarece reprezintă absența bolii, respectiv forma proliferativă. Clasele 1, 2 și 3 sunt mai apropiate clinic și vizual, iar separarea lor poate fi mai dificilă. De aceea, analiza ROC trebuie privită împreună cu matricea de confuzie, pentru a vedea dacă erorile apar mai ales între clase vecine.

6.6 Analiza matricelor de confuzie

Matricea de confuzie arată direct cum sunt distribuite predicțiile corecte și erorile. Valorile de pe diagonala principală reprezintă clasificări corecte, iar valorile din afara diagonalei arată confuziile dintre clase. Scriptul generează atât matricea absolută, cât și matricea normalizată pe clase, ceea ce ajută la interpretarea performanței indiferent de numărul exact de imagini.

În această problemă, nu toate erorile au aceeași importanță. O confuzie între clase adiacente poate reflecta caracterul gradual al bolii, în timp ce o confuzie între clasa 0 și clasa 4 ar fi mult mai gravă. Din acest motiv, analiza matricelor de confuzie este esențială pentru a evalua dacă modelul greșește într-un mod acceptabil din punct de vedere clinic sau dacă produce erori cu risc mare.

6.7 Comparație între modelele testate

Comparația dintre modele urmărește atât arhitectura, cât și funcția de pierdere. ResNet50 oferă o bază solidă, cu un raport bun între performanță și cost computațional. InceptionV3 aduce avantajul procesării multi-scală, utilă pentru leziuni de dimensiuni diferite. Inception-ResNet-V2 combină modulele Inception cu conexiuni reziduale, oferind o capacitate de reprezentare mai mare, dar cu un cost computațional crescut.

La nivelul funcțiilor de pierdere, Cross-Entropy este varianta standard și stabilă. Weighted Cross-Entropy poate fi utilă când se dorește penalizarea diferențiată a claselor. Focal Loss este potrivită pentru exemple dificile și pentru clase care se separă greu. BCE ordinal introduce ideea că stadiile bolii au o ordine naturală, dar necesită o interpretare diferită a predicțiilor.

6.8 Interpretarea erorilor de clasificare

Erorile de clasificare pot apărea din cauza calității imaginilor, a iluminării neuniforme, a artefactelor, a leziunilor foarte mici sau a asemănării dintre clasele vecine. În special, diferențele dintre retinopatia ușoară, moderată și severă pot fi subtile, iar o imagine aflată la limita dintre două stadii poate fi dificilă atât pentru model, cât și pentru evaluatorul uman.

Pentru interpretarea vizuală a deciziilor, scriptul include generarea de hărți Grad-CAM. Acestea evidențiază zonele imaginii care au contribuit cel mai mult la predicție. Dacă harta de activare se concentrează pe regiuni cu leziuni sau structuri retiniene relevante, predicția este mai ușor de justificat. Dacă activarea apare pe margini, artefacte sau zone fără semnificație clinică, rezultatul trebuie privit cu prudență.

Capitolul 7

Aplicația de inferență

7.1 Scopul aplicației dezvoltate

Pentru a demonstra utilizarea practică a modelelor antrenate, a fost dezvoltată o aplicație web de inferență. Scopul aplicației este de a permite încărcarea unei imagini de fund de ochi, selectarea unui model salvat și obținerea unei predicții privind stadiul retinopatiei diabetice. Aplicația are rol experimental și nu înlocuiește diagnosticul medical.

Rezultatul afișat include clasa prezisă, denumirea stadiului, nivelul de încredere, probabilitățile pentru fiecare clasă, timpul de inferență și recomandări orientative pentru medic și pacient. Aceste informații arată cum poate fi conectat un model antrenat la o interfață utilizabilă, nu doar la un script de evaluare.

7.2 Arhitectura aplicației

Aplicația este organizată în două componente principale: un backend Flask și un frontend React. Backend-ul este responsabil pentru încărcarea modelului, preprocesarea imaginii și calcularea predicției. Frontend-ul este responsabil pentru interfața de utilizare, selecția modelului, încărcarea imaginii și afișarea rezultatului.

Cele două componente comunică prin cereri HTTP. Backend-ul expune endpoint-uri sub forma `/api/models` și `/api/predict`, iar frontend-ul trimite cereri către acestea folosind biblioteca Axios. Pentru comunicarea între porturi diferite, backend-ul folosește `flask_cors`.

7.3 Componenta backend

Componenta backend este implementată în fișierul `Inference_site/app.py`. Aceasta definește aplicația Flask, stabilește directorul în care se află modelele salvate și selectează dispozitivul de calcul: CUDA, dacă este disponibil, sau CPU în caz contrar. Pentru reconstruirea arhitecturii modelului, backend-ul importă funcția `get_model` din `builder.py`.

Endpoint-ul `/api/models` returnează lista fișierelor `.pth` disponibile. Endpoint-ul `/api/predict` primește imaginea și numele modelului, verifică existența fișierului, aplică transformările necesare și rulează inferența. Imaginea este convertită în RGB, redimensionată la 224×224 , transformată în tensor și normalizată cu valorile ImageNet. Predicția este calculată cu `torch.no_grad()`, iar probabilitățile sunt obținute prin softmax.

7.4 Componenta frontend

Componenta frontend este implementată în `Inference_site/App.jsx`. La pornire, aceasta solicită lista de modele disponibile și selectează automat primul model găsit. Utilizatorul poate

alege un model din listă, poate încărca o imagine și poate vizualiza o previzualizare înainte de trimiterea către backend.

După apăsarea butonului de analiză, frontend-ul construiește un obiect `FormData` care conține imaginea și numele modelului selectat. Răspunsul primit de la backend este afișat în două zone: una orientată spre rezultat clinic orientativ și una cu metadata tehnice, precum timpul de inferență, dispozitivul folosit și probabilitățile brute pentru cele cinci clase.

7.5 Fluxul de utilizare

Fluxul de utilizare este simplu. Utilizatorul pornește backend-ul, deschide frontend-ul, selectează modelul dorit, încarcă o imagine de fund de ochi și apasă butonul de analiză. Imaginea este trimisă către server, unde este preprocesată și analizată de modelul ales. Rezultatul este apoi returnat în format JSON și afișat în interfața web.

Această structură permite testarea rapidă a mai multor modele pe aceeași imagine. De exemplu, o imagine poate fi analizată cu ResNet50, InceptionV3 și Inception-ResNet-V2, iar diferențele dintre probabilități pot oferi indicii despre stabilitatea predicției.

7.6 Integrarea modelului antrenat

Integrarea modelului antrenat se bazează pe reutilizarea aceleiași funcții `get_model` folosite în etapa experimentală. Fișierul `.pth` salvează ponderile, nu întreaga logică a arhitecturii, deci modelul trebuie reconstruit înainte de încărcarea parametrilor. Backend-ul identifică arhitectura prin funcția `get_base_model_name`, analizând numele fișierului selectat.

Dacă numele conține `resnet50`, se construiește un ResNet50; dacă numele conține `inception_v3`, se construiește InceptionV3; iar pentru `incres_v2`, se construiește Inception-ResNet-V2. Ponderile sunt încărcate cu `torch.load`, folosind `map_location` pentru compatibilitate cu dispozitivul disponibil. Această abordare păstrează coerența între antrenare și inferență.

7.7 Limitări practice ale aplicației

Aplicația are rol demonstrativ și prezintă mai multe limitări. În primul rând, nu verifică dacă imaginea încărcată este într-adevăr o imagine de fund de ochi sau dacă are o calitate suficientă pentru diagnostic. În al doilea rând, modelul este încărcat la fiecare cerere, ceea ce este simplu pentru testare, dar ineficient pentru producție.

Recomandările afișate sunt statice și orientative. Ele nu țin cont de istoricul pacientului, de acuitatea vizuală, de edemul macular, de tratamente existente sau de alte informații clinice. Într-o versiune reală, aplicația ar trebui să includă validare medicală, control al calității imaginilor, autentificare, audit și un mecanism de marcarea a predicțiilor incerte.

Capitolul 8

Concluzii și direcții viitoare

8.1 Concluzii generale

Lucrarea a prezentat un sistem complet pentru clasificarea automată a retinopatiei diabetice pe baza imaginilor de fund de ochi. Au fost analizate fundamentele medicale ale bolii, stadiile de severitate, particularitățile imaginilor retiniene și direcțiile relevante din literatura de specialitate. Pe această bază a fost construit un pipeline experimental în PyTorch, folosind transfer learning și arhitecturi convoluționale moderne.

Metodologia include încărcarea datelor, preprocesarea imaginilor, augmentarea pentru antrenare, construirea modelelor, alegerea funcțiilor de pierdere, antrenarea, validarea, testarea și generarea graficelor de analiză. În plus, a fost realizată o aplicație web de inferență, care demonstrează modul în care modelele salvate pot fi utilizate într-o interfață practică.

8.2 Rezultatele obținute față de obiectivele inițiale

Obiectivele inițiale au fost îndeplinite în mai multe direcții. A fost realizată o analiză a metodelor existente pentru detecția și clasificarea retinopatiei diabetice, au fost studiate seturi de date publice și a fost selectat un dataset echilibrat pentru experimente. De asemenea, a fost implementată o metodă bazată pe rețele convoluționale și transfer learning.

Codul permite testarea mai multor arhitecturi, funcții de pierdere și optimizatori. Au fost generate modele salvate, grafice de evaluare și rapoarte de clasificare. În plus, aplicația de inferență arată că rezultatele experimentale pot fi integrate într-un flux software accesibil, în care utilizatorul încarcă o imagine și primește o predicție.

8.3 Limitările lucrării

Principala limitare este faptul că modelele au fost evaluate pe un set de date public și echilibrat, care nu reflectă complet distribuția clinică reală. În practică, clasele nu sunt uniform distribuite, iar performanța pe un dataset echilibrat nu garantează automat aceeași performanță într-un program real de screening.

O altă limitare este prezența imaginilor augmentate direct în dataset. Acestea ajută la echilibrarea claselor, dar pot introduce exemple foarte asemănătoare. Pentru o validare mai strictă, ar fi utilă o împărțire la nivel de pacient, astfel încât imagini apropiate provenite de la același pacient să nu apară simultan în antrenare și testare.

Lucrarea nu include validare clinică realizată de medici oftalmologi și nu testează sistemul pe imagini provenite din mai multe centre medicale. De asemenea, aplicația de inferență este un prototip tehnic, fără autentificare, audit, stocare securizată a datelor sau certificare medicală.

8.4 Direcții viitoare

O primă direcție viitoare este validarea sistemului pe seturi de date externe, provenite din surse diferite. Această etapă ar permite evaluarea capacității de generalizare la camere, populații și protocoale de achiziție diferite. De asemenea, ar fi utilă testarea pe distribuții de date mai apropiate de practica clinică.

O altă direcție este introducerea unui modul de control al calității imaginii. Înainte de clasificare, sistemul ar putea verifica dacă fotografia este clară, bine iluminată și centrată. Cazurile neconforme ar putea fi respinse sau marcate pentru recapturare.

Pe partea de modelare, clasificarea globală poate fi completată cu detecția sau segmentarea leziunilor. Un sistem care indică nu doar stadiul bolii, ci și regiunile suspecte, ar fi mai ușor de interpretat. Aplicația de inferență poate fi extinsă prin păstrarea modelelor în memorie, calibrarea probabilităților, definirea unor praguri de incertitudine și generarea unor rapoarte exportabile.

Bibliografie

- [1] Varun Gulshan, Lily Peng, Marc Coram, Martin C Stumpe, Derek Wu, Arunachalam Narayanaswamy, Subhashini Venugopalan, Kasumi Widner, Tom Madams, Jorge Cuadros, et al. Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs. *jama*, 316(22):2402–2410, 2016.
- [2] Muhammad Kashif Jabbar, Jianzhuo Yan, Hongxia Xu, Zaka Ur Rehman, and Ayesha Jabbar. Transfer learning-based model for diabetic retinopathy diagnosis using retinal images. *Brain Sciences*, 12(5):535, 2022.
- [3] Xiaofei Wang, Mai Xu, Jicong Zhang, Lai Jiang, and Liu Li. Deep multi-task learning for diabetic retinopathy grading in fundus images. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 2826–2834, 2021.
- [4] Nitigya Sambyal, Poonam Saini, Rupali Syal, and Varun Gupta. Modified u-net architecture for semantic segmentation of diabetic retinopathy images. *Biocybernetics and Biomedical Engineering*, 40(3):1094–1109, 2020.
- [5] Shiqi Huang, Jianan Li, Yuze Xiao, Ning Shen, and Tingfa Xu. Rtnet: relation transformer network for diabetic retinopathy multi-lesion segmentation. *IEEE Transactions on Medical Imaging*, 41(6):1596–1607, 2022.
- [6] Hossein Shakibania, Sina Raoufi, Behnam Pourafkham, Hassan Khotanlou, and Muharram Mansoorizadeh. Dual branch deep learning network for detection and stage grading of diabetic retinopathy. *Biomedical Signal Processing and Control*, 93:106168, 2024.
- [7] Mohamed Chetoui and Moulay A Akhloufi. Explainable end-to-end deep learning for diabetic retinopathy detection across multiple datasets. *Journal of Medical Imaging*, 7(4):044503–044503, 2020.
- [8] Guanghua Zhang, Bin Sun, Zhaoxia Zhang, Jing Pan, Weihua Yang, and Yunfang Liu. Multi-model domain adaptation for diabetic retinopathy classification. *Frontiers in Physiology*, 13:918929, 2022.
- [9] Joes Staal, Michael D Abràmoff, Meindert Niemeijer, Max A Viergever, and Bram Van Ginneken. Ridge-based vessel segmentation in color images of the retina. *IEEE transactions on medical imaging*, 23(4):501–509, 2004.
- [10] Akara Sopharak, Bunyarit Uyyanonvara, and Sarah Barman. Automatic exudate detection from non-dilated diabetic retinopathy retinal images using fuzzy c-means clustering. *sensors*, 9(3):2148–2161, 2009.
- [11] Harry Pratt, Frans Coenen, Deborah M Broadbent, Simon P Harding, and Yalin Zheng. Convolutional neural networks for diabetic retinopathy. *Procedia computer science*, 90:200–205, 2016.
- [12] American Diabetes Association Professional Practice Committee. 2. diagnosis and classification of diabetes: Standards of care in diabetes–2024. *Diabetes Care*, 47(Supplement_1):S20–S42, 2024.

- [13] Tien Yin Wong, Chui Ming Gemmy Cheung, Michael Larsen, Sobha Sharma, and Rafael Simó. Diabetic retinopathy. *Nature Reviews Disease Primers*, 2:16012, 2016.
- [14] Luca Giancardo, Fabrice Meriaudeau, Thomas P Karnowski, Yaqin Li, Seema Garg, Kenneth W Tobin Jr, and Edward Chaum. Exudate-based diabetic macular edema detection in fundus images using publicly available datasets. *Medical image analysis*, 16(1):216–226, 2012.
- [15] Ruhan Liu, Xiangning Wang, Qiang Wu, Ling Dai, Xi Fang, Tao Yan, Jaemin Son, Shiqi Tang, Jiang Li, Zijian Gao, et al. Deepdrid: Diabetic retinopathy—grading and image quality estimation challenge. *Patterns*, 3(6), 2022.
- [16] Meindert Niemeijer, Bram Van Ginneken, Joes Staal, Maria SA Suttorp-Schulten, and Michael D Abràmoff. Automatic detection of red lesions in digital color fundus photographs. *IEEE Transactions on medical imaging*, 24(5):584–592, 2005.
- [17] Tao Li, Yingqi Gao, Kai Wang, Song Guo, Hanruo Liu, and Hong Kang. Diagnostic assessment of deep learning algorithms for diabetic retinopathy screening. *Information Sciences*, 501:511–522, 2019.
- [18] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [19] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988, 2017.
- [20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [21] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2818–2826, 2016.
- [22] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [23] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015.
- [24] Kushagra Tandon. Diabetic retinopathy balanced. Kaggle dataset, 2024. Disponibil la: <https://www.kaggle.com/datasets/kushagratandon12/diabetic-retinopathy-balanced>.

Anexa A

Cod

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import torchvision.models as models
5 import torch.optim as optim
6
7 # Importam timm pentru Inception-ResNet-v2
8 try:
9     import timm
10 except ImportError:
11     timm = None
12
13 class InceptionWrapper(nn.Module):
14     """
15     Carcasa de protectie pentru Inception-v3.
16     Lasa modelul sa se incarca nativ, dar returneaza o singura iesire la antrenare,
17     exact cum asteapta functiile noastre de Loss.
18     """
19     def __init__(self, num_classes, pretrained=True):
20         super().__init__()
21         # Lasam Inception sa se initializeze 100% standard (fara erori de aux_logits)
22         weights = models.Inception_V3_Weights.DEFAULT if pretrained else None
23         self.model = models.inception_v3(weights=weights)
24
25         # Modificam ultimul strat
26         in_features = self.model.fc.in_features
27         self.model.fc = nn.Linear(in_features, num_classes)
28
29     def forward(self, x):
30         outputs = self.model(x)
31         # La antrenare, Inception returneaza un obiect cu 2 iesiri. Noi o dam mai departe doar
32         # pe prima.
33         if self.training:
34             return outputs[0] # outputs[0] reprezinta predictia principala (logits)
35         # La validare, returneaza oricum o singura iesire
36         return outputs
37
38 # =====
39 # 1. IMPLEMENTARE CUSTOM LOSS (Focal Loss)
40 # =====
41 class FocalLossMultiClass(nn.Module):
42     """
43     Focal Loss adaptat pentru clasificare multi-clasa.
44     Accepta predictii de forma [Batch, Clase] si etichete standard (indici) de forma [Batch].
45     """
46     def __init__(self, alpha=None, gamma=1.5, reduction='mean'):
47         super(FocalLossMultiClass, self).__init__()
48         self.gamma = gamma
49         self.alpha = alpha
50         self.reduction = reduction
51
52     def forward(self, inputs, targets):
53         # Calculam direct Cross Entropy (care stie sa citeasca indicii claselor)
54         ce_loss = F.cross_entropy(inputs, targets, weight=self.alpha, reduction='none')
55         pt = torch.exp(-ce_loss)
56         focal_loss = ((1 - pt) ** self.gamma) * ce_loss
57
58         if self.reduction == 'mean':
59             return focal_loss.mean()
60         elif self.reduction == 'sum':
61             return focal_loss.sum()
62         else:

```

```

63         return focal_loss
64
65 # =====
66 # 2. FABRICA DE MODELE
67 # =====
68 def get_model(model_name, num_classes=5, pretrained=True, drop_rate=0.0):
69     """
70     Fabrica de modele. Toate au strat final modificat pentru num_classes.
71     """
72     model_name = model_name.lower()
73
74     if model_name == 'resnet50':
75         weights = models.ResNet50_Weights.DEFAULT if pretrained else None
76         model = models.resnet50(weights=weights)
77         num_fts = model.fc.in_features
78         model.fc = nn.Sequential(
79             nn.Dropout(0.5),
80             nn.Linear(num_fts, num_classes)
81         )
82
83     elif model_name == 'efficientnet_b3': # TODO adauga experiment cu asta, daca timpul
84     permite
85         weights = models.EfficientNet_B3_Weights.DEFAULT if pretrained else None
86         model = models.efficientnet_b3(weights=weights)
87         in_features = model.classifier[1].in_features
88         model.classifier[1] = nn.Sequential(
89             nn.Dropout(0.5),
90             nn.Linear(in_features, num_classes)
91         )
92
93     elif model_name == 'inception_v3':
94         model = InceptionWrapper(num_classes=num_classes, pretrained=pretrained)
95
96     elif model_name == 'incres_v2':
97         if timm is None:
98             raise ImportError(
99                 "Modelul incres_v2 necesita pachetul 'timm'. Instaleaza-l cu: pip install timm
100             )
101         model = timm.create_model(
102             'inception_resnet_v2',
103             pretrained=pretrained,
104             num_classes=num_classes,
105             drop_rate=drop_rate,
106         )
107
108     else:
109         raise ValueError(f"Modelul {model_name} nu este suportat!")
110
111     return model
112
113 # =====
114 # 3. FABRICA DE LOSS-URI SI PREDICTII (Totul se intampla aici)
115 # =====
116 def get_loss_function(loss_name, class_weights=None, device='cuda', gamma=1.5):
117     loss_name = loss_name.lower()
118
119     # -----
120     # REPARAT: Transformam lista venita din consola intr-un Tensor
121     # -----
122     if class_weights is not None and isinstance(class_weights, list):
123         class_weights = torch.tensor(class_weights, dtype=torch.float)
124
125     if loss_name in ['bce_ordinal', 'ordinal']:
126         return nn.BCEWithLogitsLoss()
127
128     elif loss_name == 'focal_loss':
129         # Trimitem ponderile pe GPU (daca exista) pentru Focal Loss
130         alpha_tensor = class_weights.to(device) if class_weights is not None else None
131         return FocalLossMultiClass(alpha=alpha_tensor, gamma=gamma)
132
133     elif loss_name == 'ce':
134         return nn.CrossEntropyLoss()
135
136     elif loss_name == 'weighted_ce':
137         if class_weights is None:

```

```

137         raise ValueError("Pentru Weighted CE, trebuie sa trimiti '--class_weights'!")
138         # Acum class_weights este oficial un Tensor, deci .to(device) va functiona perfect!
139         return nn.CrossEntropyLoss(weight=class_weights.to(device))
140
141     else:
142         raise ValueError(f"Loss-ul '{loss_name}' nu este suportat.")
143
144
145 def compute_loss(criterion, outputs, labels, loss_name, device):
146     """
147     Calculeaza valoarea erorii. Gestioneaza automat transformările necesare
148     pentru etichete (ex. transformarea in vectori ordinali).
149     """
150     loss_name = loss_name.lower()
151
152     if loss_name in ['bce_ordinal', 'ordinal']:
153         # Ordinal Regression necesita transformarea [2] -> [1, 1, 1, 0, 0]
154         levels = torch.arange(5).to(device)
155         labels_ordinal = (labels.unsqueeze(1) >= levels).float()
156         return criterion(outputs, labels_ordinal)
157
158     else:
159         # Pentru CE, Weighted CE si noul Focal Loss, putem da etichetele exact asa cum vin (
160         # indici 0-4)
161         return criterion(outputs, labels)
162
163 def get_predictions(outputs, loss_name):
164     """
165     Transforma iesirea modelului (tensor de probabilitati/logits) intr-o eticheta finala (0,
166     1, 2, 3, 4).
167     """
168     loss_name = loss_name.lower()
169
170     if loss_name in ['bce_ordinal', 'ordinal']:
171         # Adunam valorile care trec pragul de 0.0 (pragul implicit dupa BCEWithLogits)
172         preds = (outputs > 0.0).sum(dim=1) - 1
173         return torch.clamp(preds, min=0, max=4)
174
175     else:
176         # Pentru CE, Weighted CE si Focal Loss, extragem direct valoarea maxima
177         return torch.argmax(outputs, dim=1)
178
179 HEAD_PARAM_KEYWORDS = (
180     'fc',
181     'classifier',
182     'classif',
183     'head',
184     'last_linear',
185     'logits',
186 )
187
188 def is_head_parameter(name):
189     lowered = name.lower()
190     return any(keyword in lowered for keyword in HEAD_PARAM_KEYWORDS)
191
192
193 def set_backbone_trainable(model, trainable):
194     for name, param in model.named_parameters():
195         if not is_head_parameter(name):
196             param.requires_grad = trainable
197
198
199 def count_trainable_parameters(model):
200     return sum(param.numel() for param in model.parameters() if param.requires_grad)
201
202
203 def get_optimizer_params(model, lr, backbone_lr_mult=1.0):
204     if backbone_lr_mult == 1.0:
205         return model.parameters()
206
207     head_params = []
208     backbone_params = []
209     for name, param in model.named_parameters():
210         if is_head_parameter(name):

```

```

211         head_params.append(param)
212     else:
213         backbone_params.append(param)
214
215     return [
216         {'params': backbone_params, 'lr': lr * backbone_lr_mult},
217         {'params': head_params, 'lr': lr},
218     ]
219
220
221 def format_lrs(optimizer):
222     return ', '.join([f"{group['lr']:.8f}" for group in optimizer.param_groups])
223
224
225 # =====
226 # 4. FABRICA DE OPTIMIZATORI
227 # =====
228 def get_optimizer(model, optimizer_name='adam', lr=1e-4, backbone_lr_mult=1.0):
229     optimizer_name = optimizer_name.lower()
230     params = get_optimizer_params(model, lr, backbone_lr_mult=backbone_lr_mult)
231
232     if optimizer_name == 'adam':
233         return optim.Adam(params, lr=lr, weight_decay=1e-4)
234     elif optimizer_name == 'adamw':
235         return optim.AdamW(params, lr=lr, weight_decay=1e-4)
236     elif optimizer_name == 'sgd':
237         return optim.SGD(params, lr=lr, momentum=0.9, weight_decay=1e-5)
238     else:
239         raise ValueError(f"Optimizatorul '{optimizer_name}' nu este suportat.")

```

Listing A.1: builder.py – construirea modelelor, funcțiilor de pierdere și optimizatorilor

```

1 import os
2
3 import torch
4 from PIL import Image
5 from torch.utils.data import ConcatDataset, Dataset
6 import torchvision.transforms as T
7
8
9 IMAGE_EXTENSIONS = ('.png', '.jpg', '.jpeg')
10
11 EXPERIMENTS = {
12     'balanced_to_balanced': ('balanced', 'balanced'),
13     'balanced_aptos_to_balanced_aptos': ('combined', 'combined'),
14     'balanced_aptos_to_aptos': ('combined', 'aptos'),
15     'balanced_aptos_to_balanced': ('combined', 'balanced'),
16     'aptos_to_balanced': ('aptos', 'balanced'),
17     'aptos_to_balanced_aptos': ('aptos', 'combined'),
18     'balanced_to_aptos': ('balanced', 'aptos'),
19     'balanced_aug_to_balanced_aug': ('balanced_aug', 'balanced_aug'),
20     'balanced_aug_aptos_to_balanced_aug_aptos': ('balanced_aug_aptos', 'balanced_aug_aptos'),
21     'balanced_aug_aptos_to_aptos': ('balanced_aug_aptos', 'aptos'),
22     'balanced_aug_aptos_to_balanced_aug': ('balanced_aug_aptos', 'balanced_aug'),
23 }
24
25 DATA_SOURCES = [
26     'balanced',
27     'balanced_aug',
28     'aptos',
29     'combined',
30     'balanced_aptos',
31     'balanced_aug_aptos',
32     'diabetic_balanced_data+aptos',
33     'diabetic_balanced_aug+aptos',
34 ]
35
36
37 def default_transform(split='train', image_size=224, train_augment='basic'):
38     split = split.lower()
39     train_augment = train_augment.lower()
40
41     if split == 'train' and train_augment == 'basic':

```

```

42         return T.Compose([
43             T.Resize((image_size, image_size)),
44             T.RandomHorizontalFlip(p=0.5),
45             T.RandomVerticalFlip(p=0.5),
46             T.RandomRotation(degrees=15),
47             T.ColorJitter(brightness=0.1, contrast=0.1),
48             T.ToTensor(),
49             T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
50         ])
51
52     return T.Compose([
53         T.Resize((image_size, image_size)),
54         T.ToTensor(),
55         T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
56     ])
57
58
59 class BalancedDRDataset(Dataset):
60     def __init__(self, root_dir, split='train', image_size=224, transform=None, train_augment=
        'basic'):
61         self.root_dir = root_dir
62         self.split = split.lower()
63         self.image_size = image_size
64         self.split_dir = os.path.join(self.root_dir, self.split)
65
66         if not os.path.exists(self.split_dir):
67             raise FileNotFoundError(f"[Eroare] Folderul nu exista: {self.split_dir}")
68
69         self.image_paths = []
70         self.labels = []
71
72         for class_id in range(5):
73             class_folder = os.path.join(self.split_dir, str(class_id))
74             if not os.path.exists(class_folder):
75                 continue
76             for img_name in os.listdir(class_folder):
77                 if img_name.lower().endswith(IMAGE_EXTENSIONS):
78                     self.image_paths.append(os.path.join(class_folder, img_name))
79                     self.labels.append(class_id)
80
81         self.transform = transform or default_transform(self.split, self.image_size,
        train_augment=train_augment)
82
83     def __len__(self):
84         return len(self.image_paths)
85
86     def __getitem__(self, idx):
87         img_path = self.image_paths[idx]
88         label = self.labels[idx]
89
90         try:
91             image = Image.open(img_path).convert('RGB')
92         except Exception:
93             image = Image.new('RGB', (self.image_size, self.image_size), (0, 0, 0))
94
95         if self.transform:
96             image = self.transform(image)
97
98         return image, torch.tensor(label, dtype=torch.long)
99
100
101 class AptosDRDataset(BalancedDRDataset):
102     """APTOS Ben Graham este deja preprocesat si organizat pe foldere train/val/test/0..4."""
103     pass
104
105
106 def make_dataset(
107     source,
108     split,
109     image_size,
110     balanced_root,
111     aptos_root,
112     balanced_aug_root=None,
113     train_augment='basic',
114 ):
115     source = source.lower()

```

```

116     split = split.lower()
117
118     if source == 'balanced':
119         return BalancedDRDataset(balanced_root, split=split, image_size=image_size,
120                                 train_augment=train_augment)
121
122     if source == 'balanced_aug':
123         if balanced_aug_root is None:
124             raise ValueError("balanced_aug_root trebuie setat pentru sursa 'balanced_aug'.")
125         return BalancedDRDataset(balanced_aug_root, split=split, image_size=image_size,
126                                 train_augment=train_augment)
127
128     if source == 'aptos':
129         return AptosDRDataset(aptos_root, split=split, image_size=image_size, train_augment=
130                               train_augment)
131
132     if source in ('combined', 'balanced_aptos', 'diabetic_balanced_data+aptos'):
133         return ConcatDataset([
134             BalancedDRDataset(balanced_root, split=split, image_size=image_size, train_augment
135                               =train_augment),
136             AptosDRDataset(aptos_root, split=split, image_size=image_size, train_augment=
137                               train_augment),
138         ])
139
140     if source in ('balanced_aug_aptos', 'diabetic_balanced_aug+aptos'):
141         if balanced_aug_root is None:
142             raise ValueError("balanced_aug_root trebuie setat pentru sursa 'balanced_aug_aptos
143                               '.")
144         return ConcatDataset([
145             BalancedDRDataset(balanced_aug_root, split=split, image_size=image_size,
146                               train_augment=train_augment),
147             AptosDRDataset(aptos_root, split=split, image_size=image_size, train_augment=
148                               train_augment),
149         ])
150
151     raise ValueError(f"Sursa de date necunoscuta: {source}")
152
153 def resolve_experiment(experiment=None, train_source=None, test_source=None):
154     if experiment:
155         experiment = experiment.lower()
156         if experiment not in EXPERIMENTS:
157             valid = ', '.join(EXPERIMENTS.keys())
158             raise ValueError(f"Experiment necunoscut: {experiment}. Variante: {valid}")
159         return EXPERIMENTS[experiment]
160
161     return (train_source or 'balanced').lower(), (test_source or 'balanced').lower()

```

Listing A.2: my_dataset.py – încărcarea setului de date echilibrat

și generarea graficelor, și generarea graficelor,

```

1 import argparse
2 import importlib
3 import os
4 import sys
5 import time
6
7 import matplotlib.pyplot as plt
8 import numpy as np
9 import seaborn as sns
10 import torch
11 import torch.nn.functional as F
12 from sklearn.metrics import (
13     auc,
14     balanced_accuracy_score,
15     classification_report,
16     cohen_kappa_score,
17     confusion_matrix,
18     f1_score,
19     precision_recall_fscore_support,
20     roc_auc_score,
21     roc_curve,
22 )

```

```

23 from sklearn.preprocessing import label_binarize
24 from torch.utils.data import DataLoader
25 from tqdm import tqdm
26
27
28 try:
29     CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
30 except NameError:
31     CURRENT_DIR = os.getcwd()
32
33 PARENT_DIR = os.path.abspath(os.path.join(CURRENT_DIR, '..'))
34 PROJECT_ROOT = os.path.abspath(os.path.join(CURRENT_DIR, '..', '..'))
35
36 if PARENT_DIR not in sys.path:
37     sys.path.insert(0, PARENT_DIR)
38
39 from builder import (
40     compute_loss,
41     count_trainable_parameters,
42     format_lrs,
43     get_loss_function,
44     get_model,
45     get_optimizer,
46     get_predictions,
47     set_backbone_trainable,
48 )
49 import my_dataset as my_dataset_module
50
51 my_dataset_module = importlib.reload(my_dataset_module)
52 EXPERIMENTS = my_dataset_module.EXPERIMENTS
53 DATA_SOURCES = my_dataset_module.DATA_SOURCES
54 make_dataset = my_dataset_module.make_dataset
55 resolve_experiment = my_dataset_module.resolve_experiment
56
57
58 CLASS_NAMES = ['0', '1', '2', '3', '4']
59 IMAGENET_MEAN = np.array([0.485, 0.456, 0.406])
60 IMAGENET_STD = np.array([0.229, 0.224, 0.225])
61
62 RESULT_SUBDIRS = {
63     'history': 'istoric',
64     'loss': 'loss',
65     'accuracy': 'acuratete',
66     'auc': 'auc',
67     'qwk': 'qwk',
68     'macro_f1': 'macro_f1',
69     'balanced_acc': 'balanced_accuracy',
70     'confusion': 'matrici_confuzie',
71     'classification_reports': 'rapoarte_clasificare',
72     'precision_recall_f1': 'precision_recall_f1',
73     'roc': 'roc',
74     'gradcam': 'gradcam',
75     'errors': 'erori',
76     'other': 'altele',
77 }
78
79
80 def parse_args():
81     parser = argparse.ArgumentParser(description="Antrenament DR - experimente Balanced/APTOS")
82     parser.add_argument('--model', type=str, default='resnet50', help='ex: resnet50, efficientnet_b3, inception_v3')
83     parser.add_argument('--loss_name', type=str, default='ce', help='ex: ce, bce ordinal, focal_loss, weighted_ce')
84     parser.add_argument('--optimizer', type=str, default='adam', help='Variante: adam, adamw, sgd')
85     parser.add_argument('--epochs', type=int, default=20)
86     parser.add_argument('--batch_size', type=int, default=32)
87     parser.add_argument('--lr', type=float, default=0.0001)
88     parser.add_argument('--img_size', type=int, default=224)
89     parser.add_argument('--num_workers', type=int, default=0)
90     parser.add_argument('--class_weights', type=float, nargs='+', default=None)
91     parser.add_argument('--gamma', type=float, default=1.5, help='Valoarea gamma pentru Focal Loss (daca este selectat).')
92     parser.add_argument(
93         '--monitor_metric',

```

```

94         type=str,
95         default='qwk',
96         choices=['qwk', 'macro_f1', 'balanced_acc', 'auc', 'acc'],
97         help='Metrica folosita pentru checkpoint si early stopping.',
98     )
99     parser.add_argument(
100         '--scheduler',
101         type=str,
102         default='plateau',
103         choices=['plateau', 'cosine', 'none'],
104         help='Scheduler LR. plateau pastreaza comportamentul vechi; cosine e util pentru fine-
105         tuning.',
106     )
107     parser.add_argument('--amp', action='store_true', help='Activeaza mixed precision pe CUDA.
108     ')
109     parser.add_argument('--grad_clip', type=float, default=0.0, help='Clip gradient norm. 0
110     dezactiveaza.')
111     parser.add_argument('--ema_decay', type=float, default=0.0, help='EMA pentru greutati. Ex:
112     0.999. 0 dezactiveaza.')
113     parser.add_argument(
114         '--model_dropout',
115         type=float,
116         default=0.0,
117         help='Dropout intern pentru modele timm, util pentru incres_v2. 0 pastreaza
118         comportamentul implicit.',
119     )
120     parser.add_argument(
121         '--freeze_backbone_epochs',
122         type=int,
123         default=0,
124         help='Antreneaza doar head-ul in primele N epoci, apoi deblocheaza backbone-ul.',
125     )
126     parser.add_argument(
127         '--backbone_lr_mult',
128         type=float,
129         default=1.0,
130         help='Multiplicator LR pentru backbone. Ex: 0.1 inseamna backbone LR = lr * 0.1.',
131     )
132     parser.add_argument(
133         '--tta_eval',
134         action='store_true',
135         help='Activeaza TTA la validare/test pentru multiclass: original + horizontal flip.',
136     )
137     parser.add_argument(
138         '--train_augment',
139         type=str,
140         default='basic',
141         choices=['basic', 'none'],
142         help='Augmentare online pentru split-ul train. basic pastreaza comportamentul vechi;
143         none aplica doar resize/tensor/normalizare.',
144     )
145     parser.add_argument(
146         '--early_stopping_patience',
147         type=int,
148         default=0,
149         help='Opreste antrenarea dupa N epoci fara imbunatatire. 0 dezactiveaza early stopping
150         .',
151     )
152     parser.add_argument(
153         '--early_stopping_min_delta',
154         type=float,
155         default=0.0,
156         help='Imbunatatirea minima a metricii monitorizate necesara pentru resetarea patience.
157         ',
158     )
159     parser.add_argument(
160         '--train_source',
161         type=str,

```

```

162         default='balanced',
163         choices=DATA_SOURCES,
164         help='Sursa pentru train/val daca nu folosesti --experiment.',
165     )
166     parser.add_argument(
167         '--test_source',
168         type=str,
169         default='balanced',
170         choices=DATA_SOURCES,
171         help='Sursa pentru test daca nu folosesti --experiment.',
172     )
173     parser.add_argument(
174         '--test_sources',
175         type=str,
176         nargs='+',
177         default=None,
178         choices=DATA_SOURCES,
179         help='Optional: ruleaza testarea finala pe mai multe surse, in aceeasi antrenare. Ex:
--test_sources aptos balanced',
180     )
181     parser.add_argument(
182         '--root_dir',
183         type=str,
184         default=None,
185         help='Compatibilitate veche: radacina pentru Diabetic_Balanced_Data.',
186     )
187     parser.add_argument('--balanced_root', type=str, default=None)
188     parser.add_argument(
189         '--balanced_aug_root',
190         type=str,
191         default=None,
192         help='Radacina pentru Diabetic_Balanced_Aug_Ben_Graham, cu structura train/val/test
/0..4.',
193     )
194     parser.add_argument(
195         '--aptos_root',
196         type=str,
197         default=None,
198         help='Radacina APTOS deja preprocesat Ben Graham, cu structura train/val/test/0..4.',
199     )
200     return parser.parse_args()
201
202
203 def make_results_dirs(grafice_dir):
204     dirs = {}
205     os.makedirs(grafice_dir, exist_ok=True)
206     for key, folder_name in RESULT_SUBDIRS.items():
207         path = os.path.join(grafice_dir, folder_name)
208         os.makedirs(path, exist_ok=True)
209         dirs[key] = path
210     return dirs
211
212
213 # --- ADAUGAT FUNCTIA LIPSA PENTRU ISTORIC COMBINAT ---
214 def save_combined_history(history, path, show=False):
215     fig, axes = plt.subplots(2, 3, figsize=(18, 10))
216     axes = axes.ravel()
217
218     axes[0].plot(history['train_loss'], label='Train Loss')
219     axes[0].plot(history['val_loss'], label='Val Loss')
220     axes[0].set_title('Istoric Loss')
221     axes[0].set_xlabel('Epoca')
222     axes[0].set_ylabel('Loss')
223     axes[0].legend()
224
225     axes[1].plot(history['val_acc'], label='Val Accuracy', color='orange')
226     axes[1].set_title('Istoric Acuratete (Val)')
227     axes[1].set_xlabel('Epoca')
228     axes[1].set_ylabel('Acuratete')
229     axes[1].legend()
230
231     axes[2].plot(history['val_auc'], label='Val AUC', color='green')
232     axes[2].set_title('Istoric AUC (Val)')
233     axes[2].set_xlabel('Epoca')
234     axes[2].set_ylabel('AUC')
235     axes[2].legend()

```

```

236
237 axes[3].plot(history['val_qwk'], label='Val QWK', color='purple')
238 axes[3].set_title('Istoric QWK (Val)')
239 axes[3].set_xlabel('Epoca')
240 axes[3].set_ylabel('QWK')
241 axes[3].legend()
242
243 axes[4].plot(history['val_macro_f1'], label='Val Macro F1', color='red')
244 axes[4].set_title('Istoric Macro F1 (Val)')
245 axes[4].set_xlabel('Epoca')
246 axes[4].set_ylabel('Macro F1')
247 axes[4].legend()
248
249 axes[5].plot(history['val_balanced_acc'], label='Val Balanced Acc', color='brown')
250 axes[5].set_title('Istoric Balanced Accuracy (Val)')
251 axes[5].set_xlabel('Epoca')
252 axes[5].set_ylabel('Balanced Accuracy')
253 axes[5].legend()
254
255 plt.savefig(path, bbox_inches='tight')
256 if show:
257     plt.show()
258 plt.close()
259
260
261 def save_line_plot(values, title, ylabel, path, color=None, label=None, show=False):
262     plt.figure(figsize=(8, 5))
263     plt.plot(values, color=color, label=label or ylabel)
264     plt.title(title)
265     plt.xlabel('Epoca')
266     plt.ylabel(ylabel)
267     plt.legend()
268     plt.grid(alpha=0.25)
269     plt.savefig(path, bbox_inches='tight')
270     if show:
271         plt.show()
272     plt.close()
273
274
275 def save_confusion_matrix(labels, preds, path, normalized=False, show=False):
276     if normalized:
277         cm = confusion_matrix(labels, preds, labels=list(range(5)), normalize='true')
278         fmt = '.2f'
279         title = 'Matrice Confuzie Normalizata'
280     else:
281         cm = confusion_matrix(labels, preds, labels=list(range(5)))
282         fmt = 'd'
283         title = 'Matrice Confuzie'
284
285     plt.figure(figsize=(8, 6))
286     sns.heatmap(
287         cm,
288         annot=True,
289         fmt=fmt,
290         cmap='Blues',
291         xticklabels=CLASS_NAMES,
292         yticklabels=CLASS_NAMES,
293         vmin=0 if normalized else None,
294         vmax=1 if normalized else None,
295     )
296     plt.xlabel('Predictie (Model)')
297     plt.ylabel('Realitate (Adevar)')
298     plt.title(title)
299     plt.savefig(path, bbox_inches='tight')
300     if show:
301         plt.show()
302     plt.close()
303
304
305 def save_precision_recall_f1(labels, preds, base_name, results_dirs, show=False):
306     precision, recall, f1, support = precision_recall_fscore_support(
307         labels,
308         preds,
309         labels=list(range(5)),
310         zero_division=0,
311     )

```

```

312
313 x = np.arange(len(CLASS_NAMES))
314 width = 0.25
315
316 plt.figure(figsize=(10, 6))
317 plt.bar(x - width, precision, width, label='Precision')
318 plt.bar(x, recall, width, label='Recall')
319 plt.bar(x + width, f1, width, label='F1')
320 plt.xticks(x, CLASS_NAMES)
321 plt.ylim(0, 1)
322 plt.xlabel('Clase')
323 plt.ylabel('Scor')
324 plt.title('Precision / Recall / F1 pe clase')
325 plt.legend()
326 plt.grid(axis='y', alpha=0.25)
327
328 prf1_path = os.path.join(results_dirs['precision_recall_f1'], f"prf1_{base_name}.png")
329 plt.savefig(prf1_path, bbox_inches='tight')
330 if show:
331     plt.show()
332 plt.close()
333
334 report = classification_report(
335     labels,
336     preds,
337     labels=list(range(5)),
338     target_names=CLASS_NAMES,
339     zero_division=0,
340 )
341 report_path = os.path.join(results_dirs['classification_reports'], f"
classification_report_{base_name}.txt")
342 with open(report_path, 'w', encoding='utf-8') as report_file:
343     report_file.write(report)
344
345 metrics_path = os.path.join(results_dirs['precision_recall_f1'], f"prf1_{base_name}.csv")
346 with open(metrics_path, 'w', encoding='utf-8') as metrics_file:
347     metrics_file.write('class,precision,recall,f1,support\n')
348     for class_name, p, r, f, s in zip(CLASS_NAMES, precision, recall, f1, support):
349         metrics_file.write(f'{class_name},{p:.6f},{r:.6f},{f:.6f},{int(s)}\n')
350
351 return prf1_path, report_path, metrics_path, report
352
353
354 def save_roc_curve(labels, probs, base_name, results_dirs, show=False):
355     labels = np.asarray(labels)
356     probs = np.asarray(probs)
357     y_true = label_binarize(labels, classes=list(range(5)))
358
359     plt.figure(figsize=(8, 6))
360     plotted_any = False
361
362     for class_idx, class_name in enumerate(CLASS_NAMES):
363         if y_true[:, class_idx].sum() == 0:
364             continue
365         if y_true[:, class_idx].sum() == len(y_true):
366             continue
367
368         fpr, tpr, _ = roc_curve(y_true[:, class_idx], probs[:, class_idx])
369         class_auc = auc(fpr, tpr)
370         plt.plot(fpr, tpr, label=f'Clasa {class_name} (AUC={class_auc:.3f})')
371         plotted_any = True
372
373     if plotted_any:
374         try:
375             fpr_micro, tpr_micro, _ = roc_curve(y_true.ravel(), probs.ravel())
376             micro_auc = auc(fpr_micro, tpr_micro)
377             plt.plot(fpr_micro, tpr_micro, linestyle='--', color='black', label=f'Micro (AUC={
micro_auc:.3f})')
378         except Exception:
379             pass
380
381     plt.plot([0, 1], [0, 1], linestyle=':', color='gray')
382     plt.xlabel('False Positive Rate')
383     plt.ylabel('True Positive Rate')
384     plt.title('Curbe ROC multiclasa')
385     plt.legend(loc='lower right')

```

```

386     plt.grid(alpha=0.25)
387
388     roc_path = os.path.join(results_dirs['roc'], f"roc_{base_name}.png")
389     plt.savefig(roc_path, bbox_inches='tight')
390     if show:
391         plt.show()
392     plt.close()
393     return roc_path
394
395
396 def find_last_conv_layer(model):
397     last_name, last_module = None, None
398     for name, module in model.named_modules():
399         if isinstance(module, torch.nn.Conv2d):
400             last_name, last_module = name, module
401
402     if last_module is None:
403         raise RuntimeError("Nu am gasit niciun strat Conv2d pentru Grad-CAM.")
404
405     return last_name, last_module
406
407
408 def denormalize_image(tensor):
409     image = tensor.detach().cpu().permute(1, 2, 0).numpy()
410     image = (image * IMAGENET_STD) + IMAGENET_MEAN
411     return np.clip(image, 0, 1)
412
413
414 def save_gradcam_overlay(model, data_loader, loss_name, device, base_name, results_dirs, show=
False):
415     try:
416         layer_name, target_layer = find_last_conv_layer(model)
417     except RuntimeError as error:
418         print(f"Grad-CAM sarit: {error}")
419         return None
420
421     activations = []
422     gradients = []
423
424     def forward_hook(_, __, output):
425         activations.append(output.detach())
426
427     def backward_hook(_, grad_input, grad_output):
428         gradients.append(grad_output[0].detach())
429
430     forward_handle = target_layer.register_forward_hook(forward_hook)
431     backward_handle = target_layer.register_full_backward_hook(backward_hook)
432
433     model.eval()
434     try:
435         imgs, labels = next(iter(data_loader))
436         img = imgs[:1].to(device)
437         label = labels[0].item()
438
439         model.zero_grad(set_to_none=True)
440         outputs = model(img)
441         preds = get_predictions(outputs, loss_name)
442         pred = preds[0].item()
443         target_class = pred
444         score = outputs[0, target_class]
445         score.backward()
446
447         activation = activations[-1][0]
448         gradient = gradients[-1][0]
449         weights = gradient.mean(dim=(1, 2), keepdim=True)
450         cam = torch.sum(weights * activation, dim=0)
451         cam = F.relu(cam)
452         cam = cam - cam.min()
453         cam = cam / (cam.max() + 1e-8)
454         cam = F.interpolate(
455             cam.unsqueeze(0).unsqueeze(0),
456             size=img.shape[-2:],
457             mode='bilinear',
458             align_corners=False,
459         ).squeeze().cpu().numpy()
460

```

```

461     image = denormalize_image(imgs[0])
462     heatmap = plt.cm.jet(cam)[..., :3]
463     overlay = np.clip((0.55 * image) + (0.45 * heatmap), 0, 1)
464
465     gradcam_path = os.path.join(results_dirs['gradcam'], f"gradcam_{base_name}.png")
466     plt.figure(figsize=(10, 4))
467     plt.subplot(1, 2, 1)
468     plt.imshow(image)
469     plt.title(f'Original | Real: {label}')
470     plt.axis('off')
471
472     plt.subplot(1, 2, 2)
473     plt.imshow(overlay)
474     plt.title(f'Grad-CAM | Pred: {pred} | Layer: {layer_name}')
475     plt.axis('off')
476
477     plt.savefig(gradcam_path, bbox_inches='tight')
478     if show:
479         plt.show()
480     plt.close()
481     return gradcam_path
482 except Exception as error:
483     print(f"Grad-CAM sarit: {error}")
484     return None
485 finally:
486     forward_handle.remove()
487     backward_handle.remove()
488
489 class ModelEMA:
490     def __init__(self, model, decay):
491         self.decay = decay
492         self.shadow = {
493             name: param.detach().clone()
494             for name, param in model.state_dict().items()
495             if torch.is_floating_point(param)
496         }
497         self.backup = {}
498
499     @torch.no_grad()
500     def update(self, model):
501         model_state = model.state_dict()
502         for name, shadow_param in self.shadow.items():
503             shadow_param.mul_(self.decay).add_(model_state[name].detach(), alpha=1.0 - self.
504             decay)
505
506     def apply_shadow(self, model):
507         self.backup = {}
508         model_state = model.state_dict()
509         for name, shadow_param in self.shadow.items():
510             self.backup[name] = model_state[name].detach().clone()
511             model_state[name].copy_(shadow_param)
512
513     def restore(self, model):
514         model_state = model.state_dict()
515         for name, backup_param in self.backup.items():
516             model_state[name].copy_(backup_param)
517         self.backup = {}
518
519     def get_multiclass_outputs(model, imgs, use_tta=False):
520         outputs = model(imgs)
521         if not use_tta:
522             return outputs
523
524         flipped_outputs = model(torch.flip(imgs, dims=[3]))
525         return (outputs + flipped_outputs) / 2.0
526
527     def compute_extra_metrics(labels, preds):
528         try:
529             qwk = cohen_kappa_score(labels, preds, weights='quadratic')
530         except Exception:
531             qwk = 0.0
532
533     try:
534

```

```

535         macro_f1 = f1_score(labels, preds, labels=list(range(5)), average='macro',
536 zero_division=0)
537     except Exception:
538         macro_f1 = 0.0
539     try:
540         balanced_acc = balanced_accuracy_score(labels, preds)
541     except Exception:
542         balanced_acc = 0.0
543     return qwk, macro_f1, balanced_acc
544
545 def evaluate(model, data_loader, criterion, loss_name, device, desc, use_tta=False):
546     model.eval()
547     total_loss, correct, total = 0.0, 0, 0
548     all_preds, all_labels, all_probs = [], [], []
549     use_tta = use_tta and loss_name.lower() not in ['bce_ordinal', 'ordinal']
550
551     with torch.no_grad():
552         progress = tqdm(data_loader, desc=desc, leave=False)
553         for imgs, labels in progress:
554             imgs, labels = imgs.to(device), labels.to(device)
555             outputs = model(imgs)
556             loss = compute_loss(criterion, outputs, labels, loss_name, device)
557             metric_outputs = get_multiclass_outputs(model, imgs, use_tta=use_tta)
558
559             total_loss += loss.item()
560             preds = get_predictions(metric_outputs, loss_name)
561             correct += (preds == labels).sum().item()
562             total += labels.size(0)
563
564             all_preds.extend(preds.cpu().numpy())
565             all_labels.extend(labels.cpu().numpy())
566             all_probs.extend(F.softmax(metric_outputs, dim=1).cpu().numpy())
567
568     avg_loss = total_loss / max(len(data_loader), 1)
569     acc = correct / max(total, 1)
570     try:
571         auc = roc_auc_score(all_labels, all_probs, multi_class='ovr', average='macro')
572     except Exception:
573         auc = 0.0
574     qwk, macro_f1, balanced_acc = compute_extra_metrics(all_labels, all_preds)
575
576     return avg_loss, acc, auc, qwk, macro_f1, balanced_acc, all_preds, all_labels, all_probs
577
578 def select_metric(metrics, monitor_metric):
579     return {
580         'acc': metrics['acc'],
581         'auc': metrics['auc'],
582         'qwk': metrics['qwk'],
583         'macro_f1': metrics['macro_f1'],
584         'balanced_acc': metrics['balanced_acc'],
585     }[monitor_metric]
586
587
588 def make_result_base_name(base_name, test_source, use_test_suffix):
589     if not use_test_suffix:
590         return base_name
591     safe_test_source = test_source.replace('+', '_').replace('/', '_').replace('\\', '_')
592     return f"{base_name}_test_{safe_test_source}"
593
594
595 def run_final_test(
596     model,
597     test_source,
598     test_loader,
599     criterion,
600     args,
601     device,
602     base_name,
603     results_dirs,
604     use_test_suffix,
605 ):
606     result_base_name = make_result_base_name(base_name, test_source, use_test_suffix)
607
608     print("\n" + "=" * 50)

```

```

610     print(f"START TEST FINAL: {test_source}")
611     print("=" * 50)
612
613     test_loss, test_acc, test_auc, test_qwk, test_macro_f1, test_balanced_acc, test_preds,
test_labels, test_probs = evaluate(
614         model,
615         test_loader,
616         criterion,
617         args.loss_name,
618         device,
619         desc=f"Testare model [{test_source}]",
620         use_tta=args.tta_eval,
621     )
622
623     print("Rezultate testare finala:")
624     print(f"    Dataset test: {test_source}")
625     print(f"    Acuratete: {test_acc * 100:.2f}%")
626     print(f"    Scor AUC: {test_auc:.4f}")
627     print(f"    QWK: {test_qwk:.4f}")
628     print(f"    Macro F1: {test_macro_f1:.4f}")
629     print(f"    Bal Acc: {test_balanced_acc:.4f}")
630     print(f"    Loss: {test_loss:.4f}\n")
631
632     print("Classification report:")
633     prf1_path, report_path, metrics_path, report = save_precision_recall_f1(
634         test_labels,
635         test_preds,
636         result_base_name,
637         results_dirs,
638         show=True
639     )
640     print(report)
641
642     cm_path = os.path.join(results_dirs['confusion'], f"cm_{result_base_name}.png")
643     save_confusion_matrix(test_labels, test_preds, cm_path, normalized=False, show=True)
644
645     cm_norm_path = os.path.join(results_dirs['confusion'], f"cm_norm_{result_base_name}.png")
646     save_confusion_matrix(test_labels, test_preds, cm_norm_path, normalized=True, show=True)
647
648     roc_path = save_roc_curve(test_labels, test_probs, result_base_name, results_dirs, show=
True)
649     gradcam_path = save_gradcam_overlay(model, test_loader, args.loss_name, device,
result_base_name, results_dirs, show=True)
650
651     print(f"Rezultate pentru test={test_source}:")
652     print(f"Matrice confuzie: {cm_path}")
653     print(f"Matrice confuzie normalizata: {cm_norm_path}")
654     print(f"Precision/Recall/F1: {prf1_path}")
655     print(f"Raport clasificare: {report_path}")
656     print(f"Metrici CSV: {metrics_path}")
657     print(f"ROC curve: {roc_path}")
658     if gradcam_path:
659         print(f"Grad-CAM overlay: {gradcam_path}")
660
661     return {
662         'source': test_source,
663         'loss': test_loss,
664         'acc': test_acc,
665         'auc': test_auc,
666         'qwk': test_qwk,
667         'macro_f1': test_macro_f1,
668         'balanced_acc': test_balanced_acc,
669         'cm_path': cm_path,
670         'cm_norm_path': cm_norm_path,
671         'prf1_path': prf1_path,
672         'report_path': report_path,
673         'metrics_path': metrics_path,
674         'roc_path': roc_path,
675         'gradcam_path': gradcam_path,
676     }
677
678
679 def main():
680     args = parse_args()
681     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
682

```

```

683 balanced_root = args.balanced_root or args.root_dir or os.path.join(PROJECT_ROOT, '
datasets', 'Diabetic_Balanced_Data')
684 balanced_aug_root = args.balanced_aug_root or os.path.join(PROJECT_ROOT, 'datasets', '
Diabetic_Balanced_Aug_Ben_Graham')
685 aptos_root = args.aptos_root or os.path.join(PROJECT_ROOT, 'datasets', 'aptos', '
aptos_ben_graham_matched') # sau aptos_ben_graham
686 train_source, test_source = resolve_experiment(args.experiment, args.train_source, args.
test_source)
687 test_sources = args.test_sources or [test_source]
688 val_source = train_source
689 experiment_name = args.experiment or f"{train_source}_to_{test_source}"
690
691 rezultate_dir = os.path.join(CURRENT_DIR, "Rezultate")
692 modele_dir = os.path.join(rezultate_dir, "modele")
693 grafice_dir = os.path.join(rezultate_dir, "grafice")
694 os.makedirs(modele_dir, exist_ok=True)
695 results_dirs = make_results_dirs(grafice_dir)
696
697 print(f"Config: {args.model} | Loss: {args.loss_name} | Opt: {args.optimizer} | LR: {args.
lr}")
698 print(f"Experiment: {experiment_name} | TRAIN/VAL={train_source} | TEST={' ', '.join(
test_sources)}")
699 print(f"Train augment: {args.train_augment}")
700 print(f"Monitor metric: {args.monitor_metric} | Scheduler: {args.scheduler}")
701 print(f"AMP: {args.amp} | Grad clip: {args.grad_clip} | EMA decay: {args.ema_decay} | TTA
eval: {args.tta_eval}")
702 print(
703     f"Model dropout: {args.model_dropout} | Freeze backbone epochs: {args.
freeze_backbone_epochs} | "
704     f"Backbone LR mult: {args.backbone_lr_mult}"
705 )
706 if args.model.lower() == 'incres_v2' and args.img_size < 299:
707     print(
708         "Atentie: incres_v2 este de obicei folosit cu input >= 299. "
709         "Pentru rularea principala ia in calcul --img_size 299 sau 384."
710     )
711 print(f"Balanced root: {balanced_root}")
712 print(f"Balanced Aug root: {balanced_aug_root}")
713 print(f"APTOS root: {aptos_root}")
714
715 train_ds = make_dataset(
716     train_source,
717     split='train',
718     image_size=args.img_size,
719     balanced_root=balanced_root,
720     aptos_root=aptos_root,
721     balanced_aug_root=balanced_aug_root,
722     train_augment=args.train_augment,
723 )
724 val_ds = make_dataset(
725     val_source,
726     split='val',
727     image_size=args.img_size,
728     balanced_root=balanced_root,
729     aptos_root=aptos_root,
730     balanced_aug_root=balanced_aug_root,
731     train_augment=args.train_augment,
732 )
733 test_datasets = {}
734 for current_test_source in test_sources:
735     test_datasets[current_test_source] = make_dataset(
736         current_test_source,
737         split='test',
738         image_size=args.img_size,
739         balanced_root=balanced_root,
740         aptos_root=aptos_root,
741         balanced_aug_root=balanced_aug_root,
742         train_augment=args.train_augment,
743     )
744
745 train_loader = DataLoader(train_ds, batch_size=args.batch_size, shuffle=True, num_workers=
args.num_workers)
746 val_loader = DataLoader(val_ds, batch_size=args.batch_size, shuffle=False, num_workers=
args.num_workers)
747 test_loaders = {

```

```

748     current_test_source: DataLoader(test_ds, batch_size=args.batch_size, shuffle=False,
num_workers=args.num_workers)
749     for current_test_source, test_ds in test_datasets.items()
750 }
751
752 # Daca avem focal loss, afisam valoarea gamma
753 if args.loss_name == 'focal_loss':
754     print(f"Focal Loss activat cu gamma={args.gamma}")
755 test_sizes = ' | '.join([f"TESTARE[{source}]=[{len(dataset)}]" for source, dataset in
test_datasets.items()])
756 print(f"Distributie date: TRAIN=[len(train_ds)] | VALIDARE=[len(val_ds)] | {test_sizes}")
757
758 model = get_model(args.model, num_classes=5, drop_rate=args.model_dropout).to(device)
759 if args.freeze_backbone_epochs > 0:
760     set_backbone_trainable(model, trainable=False)
761     print(
762         f"Backbone inghetat pentru primele {args.freeze_backbone_epochs} epoci. "
763         f"Parametri antrenabili initial: {count_trainable_parameters(model):,}"
764     )
765 criterion = get_loss_function(args.loss_name, class_weights=args.class_weights, device=
device, gamma=args.gamma)
766 optimizer = get_optimizer(
767     model,
768     optimizer_name=args.optimizer,
769     lr=args.lr,
770     backbone_lr_mult=args.backbone_lr_mult,
771 )
772 if args.scheduler == 'plateau':
773     scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor
=0.5, patience=3)
774 elif args.scheduler == 'cosine':
775     scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=max(args.
epochs, 1), eta_min=args.lr * 0.01)
776 else:
777     scheduler = None
778 use_amp = args.amp and device.type == 'cuda'
779 scaler = torch.amp.GradScaler(device.type, enabled=use_amp)
780 ema = ModelEMA(model, args.ema_decay) if args.ema_decay > 0 else None
781
782 history = {
783     'train_loss': [],
784     'val_loss': [],
785     'val_acc': [],
786     'val_auc': [],
787     'val_qwk': [],
788     'val_macro_f1': [],
789     'val_balanced_acc': [],
790 }
791 best_metric = -float('inf')
792 epochs_without_improvement = 0
793 stopped_early = False
794 stopped_epoch = None
795 base_name = f"{experiment_name}_{args.model}_{args.loss_name}_{args.optimizer}_LR_{args.lr
}_Gamma_{args.gamma}"
796 model_path = os.path.join(modele_dir, f"best_{base_name}.pth")
797
798 for epoch in range(args.epochs):
799     start_time = time.time()
800     if args.freeze_backbone_epochs > 0 and epoch == args.freeze_backbone_epochs:
801         set_backbone_trainable(model, trainable=True)
802         print(
803             f"Backbone deblocat la epoca {epoch + 1}. "
804             f"Parametri antrenabili: {count_trainable_parameters(model):,}"
805         )
806     model.train()
807     train_loss = 0.0
808
809     progress = tqdm(train_loader, desc=f"Ep {epoch + 1:02d} [TRAIN]", leave=False)
810     for imgs, labels in progress:
811         imgs, labels = imgs.to(device), labels.to(device)
812         optimizer.zero_grad(set_to_none=True)
813         with torch.amp.autocast(device.type, enabled=use_amp):
814             outputs = model(imgs)
815             loss = compute_loss(criterion, outputs, labels, args.loss_name, device)
816
817         scaler.scale(loss).backward()

```

```

818         if args.grad_clip > 0:
819             scaler.unscale_(optimizer)
820             torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
821             scaler.step(optimizer)
822             scaler.update()
823         if ema is not None:
824             ema.update(model)
825
826         train_loss += loss.item()
827         progress.set_postfix({'loss': f"{loss.item():.4f}"})
828
829         epoch_train_loss = train_loss / max(len(train_loader), 1)
830         if ema is not None:
831             ema.apply_shadow(model)
832         val_loss, val_acc, val_auc, val_qwk, val_macro_f1, val_balanced_acc, _, _, _ =
evaluate(
833             model,
834             val_loader,
835             criterion,
836             args.loss_name,
837             device,
838             desc=f"Ep {epoch + 1:02d} [VALID]",
839             use_tta=args.tta_eval,
840         )
841         if ema is not None:
842             ema.restore(model)
843
844         history['train_loss'].append(epoch_train_loss)
845         history['val_loss'].append(val_loss)
846         history['val_acc'].append(val_acc)
847         history['val_auc'].append(val_auc)
848         history['val_qwk'].append(val_qwk)
849         history['val_macro_f1'].append(val_macro_f1)
850         history['val_balanced_acc'].append(val_balanced_acc)
851
852         metrics = {
853             'acc': val_acc,
854             'auc': val_auc,
855             'qwk': val_qwk,
856             'macro_f1': val_macro_f1,
857             'balanced_acc': val_balanced_acc,
858         }
859         metric = select_metric(metrics, args.monitor_metric)
860         if scheduler is not None:
861             if args.scheduler == 'plateau':
862                 scheduler.step(metric)
863             else:
864                 scheduler.step()
865         marker = ""
866         if metric > best_metric + args.early_stopping_min_delta:
867             best_metric = metric
868             epochs_without_improvement = 0
869             if ema is not None:
870                 ema.apply_shadow(model)
871             torch.save(model.state_dict(), model_path)
872             if ema is not None:
873                 ema.restore(model)
874             marker = "NOU BEST"
875         else:
876             epochs_without_improvement += 1
877
878         print(
879             f"Ep {epoch + 1:02d}/{args.epochs} | "
880             f"T_Loss: {epoch_train_loss:.4f} | V_Loss: {val_loss:.4f} | "
881             f"V_Acc: {val_acc * 100:.1f}% | V_AUC: {val_auc:.4f} | "
882             f"V_QWK: {val_qwk:.4f} | V_F1: {val_macro_f1:.4f} | "
883             f"V_BalAcc: {val_balanced_acc:.4f} | Best[{args.monitor_metric}]= {best_metric:.4f}
{marker}"
884         )
885         print(f"LR curent: {format_lrs(optimizer)}")
886         print(f"Durata epoca: {(time.time() - start_time) / 60:.2f} min")
887         if args.early_stopping_patience > 0:
888             print(
889                 f"Early stopping: {epochs_without_improvement}/"
890                 f"{args.early_stopping_patience} epoci fara imbunatatire"
891             )

```

```

892         if epochs_without_improvement >= args.early_stopping_patience:
893             stopped_early = True
894             stopped_epoch = epoch + 1
895             print(
896                 f"Early stopping activat la epoca {stopped_epoch}. "
897                 f"Cea mai buna metrica: {best_metric:.4f}"
898             )
899             break
900
901 model.load_state_dict(torch.load(model_path, map_location=device))
902
903 # Afisare grafice de acuratete, loss, AUC, matricile de confuzie (normala + normalizata),
904 # Grad-CAM, curba ROC
905 # Am adaugat parametru show=True ca sa apara pozele dupa testare
906
907 history_path = os.path.join(results_dirs['history'], f"istoric_combinat_{base_name}.png")
908 save_combined_history(history, history_path, show=True)
909
910 loss_path = os.path.join(results_dirs['loss'], f"loss_{base_name}.png")
911 save_line_plot(history['train_loss'], 'Loss Antrenament', 'Loss', loss_path, color='blue',
912                 label='Train Loss', show=True)
913
914 acc_path = os.path.join(results_dirs['accuracy'], f"acc_{base_name}.png")
915 save_line_plot(history['val_acc'], 'Acuratete Validare', 'Acuratete', acc_path, color='
916 orange', label='Val Acc', show=True)
917
918 auc_path = os.path.join(results_dirs['auc'], f"auc_{base_name}.png")
919 save_line_plot(history['val_auc'], 'Scor AUC Validare', 'AUC', auc_path, color='green',
920                 label='Val AUC', show=True)
921
922 qwk_path = os.path.join(results_dirs['qwk'], f"qwk_{base_name}.png")
923 save_line_plot(history['val_qwk'], 'Quadratic Weighted Kappa Validare', 'QWK', qwk_path,
924                 color='purple', label='Val QWK', show=True)
925
926 macro_f1_path = os.path.join(results_dirs['macro_f1'], f"macro_f1_{base_name}.png")
927 save_line_plot(history['val_macro_f1'], 'Macro F1 Validare', 'Macro F1', macro_f1_path,
928                 color='red', label='Val Macro F1', show=True)
929
930 balanced_acc_path = os.path.join(results_dirs['balanced_acc'], f"balanced_acc_{base_name}.
931 png")
932 save_line_plot(
933     history['val_balanced_acc'],
934     'Balanced Accuracy Validare',
935     'Balanced Accuracy',
936     balanced_acc_path,
937     color='brown',
938     label='Val Balanced Acc',
939     show=True,
940 )
941
942 use_test_suffix = len(test_loaders) > 1
943 test_results = []
944 for current_test_source, current_test_loader in test_loaders.items():
945     test_results.append(run_final_test(
946         model=model,
947         test_source=current_test_source,
948         test_loader=current_test_loader,
949         criterion=criterion,
950         args=args,
951         device=device,
952         base_name=base_name,
953         results_dirs=results_dirs,
954         use_test_suffix=use_test_suffix,
955     ))
956
957 print(f"\nGata. Model: {model_path}")
958 print(f"Grafic evolutie combinat: {history_path}")
959 print(f"Grafic loss: {loss_path}")
960 print(f"Grafic acuratete: {acc_path}")
961 print(f"Grafic AUC: {auc_path}")
962 print(f"Grafic QWK: {qwk_path}")
963 print(f"Grafic Macro F1: {macro_f1_path}")
964 print(f"Grafic Balanced Acc: {balanced_acc_path}")
965
966 if stopped_early:
967     print(f"Antrenare oprita anticipat la epoca {stopped_epoch}/{args.epochs}.")
968 if len(test_results) > 1:

```

```

961     print("Rezumat testari finale:")
962     for result in test_results:
963         print(
964             f"    {result['source']}: "
965             f"Acc={result['acc'] * 100:.2f}% | AUC={result['auc']:.4f} | "
966             f"QWK={result['qwk']:.4f} | F1={result['macro_f1']:.4f} | "
967             f"BalAcc={result['balanced_acc']:.4f} | Loss={result['loss']:.4f}"
968         )
969
970
971 if __name__ == '__main__':
972     if not hasattr(sys.modules['__main__'], '__spec__'):
973         sys.modules['__main__'].__spec__ = None
974     main()

```

Listing A.3: train.py – antrenarea